



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра инструментального и прикладного программного обеспечения

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой _____ Р.Г. Болбаков

« 01 » _____ 2024 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки бакалавров 09.03.04 Программная инженерия

На тему: Веб-сервис для создания, управления и отслеживания спортивных соревнований
между искусственным интеллектом

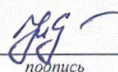
Обучающийся


подпись

Жизневский Никита Вячеславович
Фамилия, имя, отчество

шифр 20И1914
группа ИКБО-16-20

Руководитель
работы


подпись

к.т.н., доцент,
доцент

Стариковская Н.А.

Консультант


подпись

к.э.н., доцент

Жемерикин О.И.

Москва 2024 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий

Кафедра инструментального и прикладного программного обеспечения

СОГЛАСОВАНО

Заведующий
кафедрой

подпись

Болбаков Роман Геннадьевич

«14»

февраля

2024 г.

УТВЕРЖДАЮ

Директор
института

подпись

Зуев Андрей Сергеевич

«14»

февраля

2024 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

Обучающийся

Жизневский Никита Вячеславович

Фамилия Имя Отчество

Шифр

20И1914

Направление
подготовки

09.03.04

индекс направления

Программная инженерия

наименование направления

Группа

ИКБО-16-20

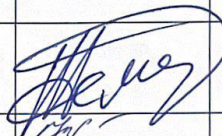
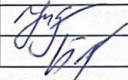
1. Тема выпускной квалификационной работы: Веб-сервис для создания, управления и отслеживания спортивных соревнований между искусственным интеллектом

2. Цель и задачи выпускной квалификационной работы

Цель работы: спроектировать и разработать веб-сервис для создания, управления и отслеживания спортивных соревнований между искусственным интеллектом

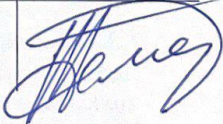
Задачи работы: провести анализ предметной области – существующих конкурентных программных решений; сформулировать основные требования к веб-сервису; выбрать средства разработки клиентской и серверной части; спроектировать клиент-серверное веб-приложение (архитектуру, функциональную схему, адаптированную модель жизненного цикла, интерфейс и базу данных); разработать клиент-серверное веб-приложение; выполнить тестирование приложения на соответствие требованиям; рассчитать экономическую эффективность и стоимость проведения работ; оформить пояснительную записку согласно ГОСТ 7.32-2017

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Аналитический раздел		21.03.2024
1.1	Анализ конкурентных решений		
1.2	Выбор средств разработки клиентской части		
1.3	Выбор средств разработки серверной части		
1.4	Требования к разработке веб-сервиса		
2	Проектный раздел		22.03.2024
2.1	Проектирование функциональной схемы веб-сервиса		
2.2	Проектирование адаптированной модели жизненного цикла		
2.3	Проектирование базы данных		
2.4	Проектирование архитектуры клиентской части		
2.5	Проектирование архитектуры серверной части		
3	Технологический раздел		19.04.2024
3.1	Разработка бизнес-логики приложения		
3.2	Разработка клиент-серверного приложения		
3.2.1	Разработка клиентской части		
3.2.2	Разработка серверной части		
3.3	Тестирование приложения на соответствие требованиям		
4	Экономический раздел		17.05.2024
4.1	Организация и планирование работ по теме		
4.2	Расчет стоимости проведения работ по теме		17.05.2024
5	Введение, заключение, список источников, приложения		
6	Презентационный материал, аннотация		24.05.2024
7	Нормоконтроль		25.05.2024

4. Перечень разрабатываемых документов и графических материалов: электронная версия выпускной квалификационной работы бакалавра, презентационный материал с основными результатами выпускной квалификационной работы бакалавра

5. Руководитель и консультанты выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	доцент	Стариковская Надежда Анатольевна	
Консультант по экономическому разделу	доцент	Жемерикин Олег Игоревич	

Задание выдал

Руководитель ВКР: _____


подпись

«14» февраля 2024 г.

Задание принял к исполнению

Обучающийся: _____


подпись

«14» февраля 2024 г.

УДК 004.4

Руководитель ВКР: к.т.н., доцент, Стариковская Н.А.

Консультант по экономическому разделу: к.э.н., доцент Жемерикин О.И.

Жизневский Н.В. Выпускная квалификационная работа по образовательной программе «Разработка программных продуктов и проектирование информационных систем» направления подготовки «Программная инженерия» на тему «Веб-сервис для создания, управления и отслеживания спортивных соревнований между искусственным интеллектом»: М. 2024 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО) – 75 стр., 36 рис., 9 табл., 39 источн. , 6 прил.

Ключевые слова: *искусственный интеллект, Java, Spring, веб-сервис, соревнования.*

Целью работы является проектирование и разработка клиент-серверного веб-приложения для создания, управления и отслеживания спортивных соревнований между искусственным интеллектом.

Zhiznevsky N.V. Final qualifying work for the educational program “Development of software products and design of information systems” in the direction of training “Software Engineering” on the topic “Web service for creating, managing and tracking sports competitions between artificial intelligence”: M. 2024, MIREA - Russian Technological University University (RTU MIREA), Institute of Information Technologies (IIT), Department of Instrumental and Application Software (IiPPO) – 75 pages, 36 figures, 9 tables, 39 sources., 6 adj.

Keywords: artificial intelligence, Java, Spring, web service, competitions.

The goal of the work is to design and develop a client-server web application for creating, managing and tracking sports competitions between artificial intelligence.

РТУ МИРЭА: 119454, Москва, пр-т Вернадского, д. 78

кафедра инструментального и прикладного программного обеспечения (ИиППО)

Тираж: 1 экз. (на правах рукописи)

Файл: «090304_20И1914_Жизневский НВ.docx», исполнитель Жизневский Н.В.

© Н.В. Жизневский

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ.....	8
1 Анализ предметной области	11
1.1 Анализ конкурентных решений	11
1.1.1 Яндекс афиша.....	11
1.1.2 Мос.ру афиша.....	12
1.1.3 Сравнение выбранных аналогов.....	12
1.2 Выбор средств разработки клиентской части	14
1.2.1 React.....	14
1.3 Выбор средств разработки серверной части	15
1.3.1 Java.....	15
1.3.2 Spring Boot	16
1.4 Требования к разработке веб-сервиса.....	17
1.5 Выводы к разделу 1.....	19
2 Проектный раздел	20
2.1 Проектирование функциональной схемы веб-сервиса	20
2.2 Проектирование адаптированной модели жизненного цикла.....	23
2.3 Проектирование базы данных	24
2.4 Проектирование архитектуры клиентской части	27
2.5 Проектирование архитектуры серверной части	28
2.6 Выводы к разделу 2.....	31
3 Технологический раздел.....	32
3.1 Разработка бизнес-логики приложения	32
3.2 Разработка клиент-серверного приложения.....	32

3.2.1	Разработка клиентской части.....	33
3.2.2	Разработка серверной части.....	34
3.3	Тестирование приложения на соответствие требованиям.....	38
3.4	Расчет вычислительной и ёмкостной сложностей	41
3.5	Выводы к разделу 3.....	43
4	Экономический раздел	44
4.1	Организация и планирование работ.....	44
4.2	Расчёт затрат на проведение работ	46
4.2.1	Расчет основной заработной платы	46
4.2.2	Вычисление дополнительной заработной платы.....	48
4.2.3	Страховые взносы	49
4.2.4	Материалы, покупные изделия и полуфабрикаты.....	49
4.2.5	Расходы на приобретение компьютерной техники и программного обеспечения	50
4.2.6	Амортизационные отчисления	50
4.2.7	Эксплуатационные расходы, связанные с использованием компьютерной техники.....	51
4.2.8	Накладные расходы	51
4.2.9	Прочие расходы.....	51
4.2.10	Калькуляция затрат по проекту	52
4.3	Выводы к разделу 4.....	52
	ЗАКЛЮЧЕНИЕ	53
	СПИСОК ИСТОЧНИКОВ.....	55
	ПРИЛОЖЕНИЕ А	59
	ПРИЛОЖЕНИЕ Б.....	60

ПРИЛОЖЕНИЕ В	62
ПРИЛОЖЕНИЕ Г	64
ПРИЛОЖЕНИЕ Д	70
ПРИЛОЖЕНИЕ Е.....	74

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

БД	– база данных
ГОСТ	– межгосударственный стандарт
ИИ	– искусственный интеллект
ОЗУ	– оперативное запоминающее устройство
ОС	– операционная система
ПО	– программное обеспечение
СУБД	– система управления базами данных
API	– Application Programming Interface, описание способов взаимодействия одной компьютерной программы с другими
CASE	– computer-aided software engineering, набор инструментов и методов программной инженерии для проектирования программного обеспечения
JWT	– JSON Web Token, это открытый стандарт (RFC 7519) для создания токенов доступа, основанный на формате JSON
HTTP	– HyperText Transfer Protocol, (протокол передачи гипертекста)
JSON	– JavaScript Object Notation, текстовый формат обмена данными, основанный на JavaScript
MVC	– Model-View-Controller, Модель-Представление-Контроллер
REST	– Representational State Transfer, «передача репрезентативного состояния», архитектурный стиль взаимодействия компонентов распределённого приложения в сети

ВВЕДЕНИЕ

Спорт и технологии искусственного интеллекта (ИИ) – две области, которые, казалось бы, крайне отдалены друг от друга. Однако, с развитием современных технологий, границы между ними становятся все более размытыми. В последние десятилетия ИИ проделал огромный путь в различных областях, от медицины до финансов, и спорт не стал исключением.

Актуальность темы подтверждается стремительным развитием технологий искусственного интеллекта и их влиянием на различные сферы жизни, включая спорт. Системы искусственного интеллекта демонстрируют потенциал в анализе данных, принятии решений и прогнозировании результатов, что делает их важными участниками в современных спортивных состязаниях. Такой подход не только стимулирует развитие технологий, но и открывает новые возможности для улучшения стратегий и тактики в спорте.

Объект исследования – веб-сервис для создания, управления и отслеживания спортивных соревнований между искусственным интеллектом.

Предмет исследования – веб-сервис, предоставляющий услуги по организации и участию пользователей в спортивных соревнованиях между искусственным интеллектом.

Теоретико-методологическую базу проекта составляют учебные пособия, справочники, ресурсы в сети Интернет.

Новизна исследования заключается в том, что предложенный веб-сервис представляет собой уникальную платформу, сфокусированную исключительно на спортивных соревнованиях между искусственным интеллектом. До этого момента подобные решения не были широко представлены на рынке спортивных и технологических платформ.

Целью данной веб-платформы является создание универсального ресурса, где разработчики, спортсмены и спортивные организации могут находить и участвовать в соревнованиях, проводимых между ИИ-агентами. Платформа предоставляет возможность как для участия в состязаниях, так и

для их организации и управления, что в свою очередь способствует развитию как искусственного интеллекта, так и спортивной индустрии в целом.

Для достижения цели планируется выполнить следующий перечень задач:

1. Провести анализ предметной области – существующих конкурентных программных решений.
2. Сформулировать основные требования к веб-сервису.
3. Выбрать средства разработки клиентской и серверной части.
4. Спроектировать клиент-серверное веб-приложение (архитектуру, функциональную схему, адаптированную модель жизненного цикла, интерфейс и базу данных).
5. Разработать клиент-серверное веб-приложение.
6. Выполнить тестирование приложения на соответствие требованиям.
7. Рассчитать вычислительную и емкостную сложности.
8. Рассчитать экономическую эффективность и стоимость проведения работ.
9. Оформить пояснительную записку согласно ГОСТ 7.32-2017.

В процессе написания выпускной квалификационной работы автор руководствовался следующими нормативными актами:

1. «О защите населения и территории от чрезвычайных ситуаций природного и техногенного характера» от 21.12.1994 № 68-ФЗ.
2. «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ.
3. «О гражданской обороне» от 12.02.1998 № 28-ФЗ.
4. Приказ Минздравсоцразвития РФ от 04.05.2012 № 477н «Об утверждении перечня состояний, при которых оказывается первая помощь, и перечня мероприятий по оказанию первой помощи».
5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ.

6. СанПиН 2.2.2/2.4.1340-03 «Гигиенические требования к персональным электронно-вычислительным машинам и организации работы».

Стандарты, используемые в работе: ГОСТ 7.32-2017 [1], ГОСТ Р 7.0.100-2018 [2], СМК О МИРЭА 7.5.1/03.П.30-19 [3], СМК О МИРЭА 7.5.1/03.П.67-19 [4], ФГОС ВО по направлению подготовки 09.03.04 Программная инженерия [5].

1 Анализ предметной области

На сегодняшний день наблюдается растущий интерес к организации спортивных соревнований между искусственным интеллектом (ИИ), что приводит к развитию специализированных платформ для агрегации и управления такими событиями.

1.1 Анализ конкурентных решений

На сегодняшний день полноценных агрегаторов спортивных соревнований между искусственным интеллектом пока не существует. Однако, можно рассмотреть аналоги агрегаторов событий в других областях, которые могут служить вдохновением или примером для разработки подобных платформ.

1.1.1 Яндекс афиша

«Яндекс.Афиша» – это онлайн-сервис, предоставляющий пользователю информацию о различных мероприятиях, событиях и развлечениях в его городе [6]. Платформа включает в себя широкий спектр развлекательных событий и программ, однако не предоставляет событий связанных с спортивными соревнованиями между искусственным интеллектом (Рисунок А.1).

Еще одним недостатком «Яндекс.Афиши» является тот факт, что регистрация на большинство культурно-массовых мероприятий является платной. Это сокращает количество потенциальных участников, что в свою очередь сказывается на популяризации данных мероприятий. Особенно платная регистрация может стать барьером для студентов и школьников, которые могут быть не готовы выделять средства на данный вид времяпрепровождения.

Следующий недостаток «Яндекс.Афиши» заключается в полном отсутствии онлайн чата пользователей. Данный недостаток приводит к тому, что пользователи не могут обмениваться личным опытом участия в конкретных соревнованиях в реальном времени. Также онлайн чат является хорошей возможностью для поиска партнеров для участия в соревнованиях.

1.1.2 Мос.ру афиша

«Мос.ру Афиша» – это веб-платформа, которая предоставляет обширную информацию о различных событиях, мероприятиях и развлечениях, происходящих в Москве и пригороде [7]. Она так же как и предыдущая платформа нацелена на предоставления людям огромного количества культурно-массовых мероприятий (Рисунок А.2).

Помимо всех недостатков прошлой обозреваемой платформы, в базе данных «Мос.ру Афиши» могут отсутствовать некоторые малоизвестные или небольшие мероприятия, особенно если они организованы независимыми или малыми структурами. Люди могут упустить возможность посетить интересное событие из-за его отсутствия на платформе.

1.1.3 Сравнение выбранных аналогов

Обе платформы предоставляют обширную информацию о различных событиях и мероприятиях, что помогает пользователям находить интересные мероприятия для проведения свободного времени (Таблица 1.1).

Однако, обе платформы не предоставляют возможности найти или зарегистрироваться на события, связанные с соревнованиями искусственного интеллекта. Это ограничение уменьшает их ценность для пользователей, интересующихся этим видом мероприятий.

Плата за регистрацию на многие события может быть барьером для многих пользователей, особенно для тех, кто не готов выделять средства на участие в мероприятиях. Это ограничение может привести к уменьшению количества участников и оказать негативное влияние на популяризацию различных событий.

Отсутствие полноценного онлайн чата для пользователей также является недостатком обеих платформ. Это ограничение снижает уровень взаимодействия и обмена мнениями между участниками, что может уменьшать ценность платформы для пользователей.

Таблица 1.1 – Сравнение приведенных аналогов [разработано автором]

Требование	Яндекс афиша	Мос.ру афиша
Просмотр списка событий	Да	Да
Поиск события	Да	Да
Бесплатность событий	Нет	Нет
Пользовательский онлайн чат	Нет	Нет
Регистрация, авторизация, аутентификация пользователя	Да	Да
События соревнований между ИИ	Нет	Нет
Модерация событий	Да	Нет
Сохранение данных в БД	Да	Да
Совместимость веб-приложения с современными браузерами	Да	Да
Пользовательский (покупательский) форум	Нет	Да
Обратная связь	Да	Да
Адаптивность дизайна интерфейса	Да	Да
Простая навигация	Нет	Нет
Привлекательность дизайна интерфейса	Да	Нет
Итого	9/14	8/14

Таким образом, для того чтобы иметь конкурентное преимущество, разрабатываемая система должна предоставлять пользователю удобный и полнофункциональный инструмент для организации, управления и участия в спортивных соревнованиях между искусственным интеллектом. Это включает в себя следующие аспекты:

- обширную базу данных событий с подробной информацией о датах, местах проведения, правилах и условиях участия; интуитивно понятный и привлекательный пользовательский интерфейс;
- возможность коммуникации и взаимодействия между участниками;
- бесплатную регистрацию на соревнования для минимизации финансовых барьеров;

- поддержку различных форматов и типов спортивных соревнований.

В целом, разрабатываемая система должна быть универсальным инструментом, который удовлетворяет потребности организаторов и участников, обеспечивая им удобство, эффективность и возможность взаимодействия.

1.2 Выбор средств разработки клиентской части

При выборе средств разработки клиентской части приложения стоит учитывать множество факторов, таких как производительность, масштабируемость и удобство разработки.

1.2.1 React

React – это JavaScript библиотека для создания пользовательских интерфейсов, разработанная компанией Facebook [8]. Его основным преимуществом является декларативный подход к созданию компонентов пользовательского интерфейса, что делает код более понятным и легко поддающимся обслуживанию. Одним из ключевых преимуществ React является использование виртуального DOM, что повышает производительность и эффективность при обновлении интерфейса.

Преимущества React:

- эффективное управление состоянием компонентов через внутренний state и жизненные циклы компонентов;
- возможность создания переиспользуемых компонентов, что способствует модульности и читаемости кода;
- обширное сообщество разработчиков и наличие множества сторонних библиотек и инструментов для улучшения разработки;
- поддержка серверного рендеринга, что улучшает SEO и производительность приложения.

Аналогом React служит Vue.js [9]. Vue.js – это прогрессивный JavaScript фреймворк, который широко используется для создания пользовательских интерфейсов. Как и React, Vue.js также обеспечивает декларативное создание

компонентов, что способствует понятности и обслуживанию кода. Он также предлагает эффективное управление состоянием компонентов и поддерживает использование виртуального DOM для оптимизации производительности.

Хотя Vue.js является мощным и популярным фреймворком, выбор между React и Vue.js был сделан в сторону первого по нескольким причинам:

- экосистема: React имеет более широкую и зрелую экосистему, включая большое количество сторонних библиотек и инструментов, что облегчает разработку и поддержку проекта;
- крупные компании: React активно поддерживается крупными компаниями, такими как Facebook, Instagram и Airbnb, что обеспечивает стабильность и долгосрочную поддержку проекта;
- гибкость: React предоставляет большую гибкость в организации кода и архитектурных решений, что позволяет легко масштабировать проекты и внедрять сложные функциональности.

1.3 Выбор средств разработки серверной части

При выборе инструментов и методов создания веб-сервиса стоит руководствоваться выбранной предметной областью, а также особенностями современных подходов разработки.

1.3.1 Java

Сервер приложений реализуется с помощью языка Java [10]. Java – типизированный объектно-ориентированный язык программирования.

Преимущества Java:

- независимость от платформы, на которой происходит выполнение написанных программ, код запускается под управлением таких ОС, как Windows, Linux, Solaris;
- Java – полностью объектно-ориентированный язык. Все сущности в Java являются объектами (за исключением немногих типов);

- возможность явного выделения памяти исключена. Освобождение памяти в Java происходит автоматически с помощью механизма сборки мусора;

- на Java реализуется получение, валидация и преобразование данных от клиентской части приложения. Данный язык является одним из самых распространенных языков для разработки серверной части веб-приложений.

Один из самых популярных аналогов Java - это язык программирования Python. Python также обладает рядом преимуществ и используется во многих областях разработки, включая веб-приложения. Ниже представлены основные преимущества Python [11].

Хотя Python является мощным и популярным языком программирования, я выбрал Java для реализации серверного приложения по следующим причинам:

- производительность: Java обычно предлагает более высокую производительность по сравнению с Python, особенно при работе с многопоточными приложениями и высоконагруженными системами;

- масштабируемость: Java лучше масштабируется для крупных проектов, особенно при использовании технологий, таких как Java Enterprise Edition (Java EE) или Spring Framework;

- статическая типизация: Java - статически типизированный язык программирования, что обеспечивает более строгую проверку типов во время компиляции и помогает предотвратить ошибки времени выполнения.

1.3.2 Spring Boot

Также важным компонентом серверной части является фреймворк Spring Boot, который упрощает использование фреймворка Spring [12]. Spring Framework – универсальный фреймворк с открытым исходным кодом для Java-платформы.

Преимущества Spring:

- нет необходимости не только создавать объекты, а также связывать их друг с другом;
- возможность разработки проекта как набора слабосвязанных моделей;
- вынесение управления зависимостями в отдельный файл облегчает дальнейшие изменения в приложении.

Один из наиболее популярных аналогов Spring Boot для разработки приложений на Java - это Micronaut [13]. Micronaut разработан компанией Object Computing и предлагает множество сходств с Spring Boot, а также некоторые уникальные особенности, однако выбор был сделан в сторону Spring Boot по следующим причинам:

- огромное сообщество и экосистема: Spring Boot имеет огромное сообщество разработчиков и обширную экосистему инструментов и библиотек, что облегчает поддержку и разработку приложений;
- широкие возможности конфигурации: Spring Boot предоставляет множество способов настройки приложений, что делает его очень гибким и настраиваемым под различные потребности;
- интеграция с другими проектами Spring: Spring Boot плотно интегрирован с другими проектами Spring, такими как Spring Security, Spring Data, Spring Cloud и т. д., что обеспечивает максимальную гибкость и функциональность для разработчиков;
- стабильность и надежность: Spring Boot имеет долгую историю разработки, он широко используется в индустрии и считается стабильным и надежным решением для разработки крупных и критически важных приложений.

1.4 Требования к разработке веб-сервиса

Сформулируем требования к разработке веб-сервисе исходя из пользовательских историй.

Пользовательские истории:

Я, как пользователь, хотел бы иметь возможность регистрации и авторизации. В случае необходимости я хотел бы изменять данные своего профиля (Имя, Фамилию, Пароль, Фотографию и тд). Также мне хотелось бы иметь возможность делиться своими положительными/негативными впечатлениями о прошедшем соревновании в онлайн чате, причем как в общем, так и в приватном. Я хотел бы просматривать список текущих соревнований (запланированных, идущих и законченных), а также просматривать список команд, с возможностью создания или вступления в определенную команду.

Организаторские истории:

Я, как организатор, хочу иметь возможность создавать соревнования и отправлять их на рассмотрение администратору, чтобы избежать проблем. Также я хочу иметь возможность начинать и завершать соревнования, а в случае необходимости — удалять их.

Администраторские истории:

Я, как администратор, хочу иметь возможность деактивации аккаунта пользователя, а также выдачи и забора привилегий пользователя. Кроме того, я хочу иметь возможность создавать аккаунты пользователей.

Модераторские истории:

Я, как модератор, хочу иметь возможность модерирования событий, созданных организаторами, чтобы обеспечить соблюдение правил и корректное проведение мероприятий.

Эти истории помогут в разработке функционала, который обеспечит удовлетворение потребностей всех участников системы, улучшая пользовательский опыт и облегчая управление соревнованиями.

На основе вышеприведенных пользовательских историй, можно сформулировать функциональные требования к системе:

- авторизация/аутентификация/регистрация;
- личный кабинет с возможностью смены данных и аватара;

- пользовательский чат с публичной и приватными комнатами;
- страница соревнований с возможностью зарегистрировать команду на соревнование;
- страница команд с возможностью создать или вступить в команду;
- страница администратора с возможностью выдачи/забора прав у пользователя, а также возможностью создания аккаунта пользователя;
- дополнительные действия администратора на странице соревнований с возможностью создания и отправки на модерацию событий, а также удаление созданных событий;
- страница модератора для возможности принять/отклонить созданное ранее соревнование организатором.

1.5 Выводы к разделу 1

Анализ предметной области показал нехватку специализированных платформ для организации соревнований между искусственным интеллектом. Основные проблемы существующих аналогов включают отсутствие бесплатной регистрации и онлайн-чата. Были сформулированы требования к программному продукту, направленные на создание удобного инструмента для управления соревнованиями, а также выбраны соответствующие технологии для его реализации.

2 Проектный раздел

2.1 Проектирование функциональной схемы веб-сервиса

В проектируемом веб-сервисе, пользователь, попадая на главную страницу может как авторизоваться, так и зарегистрироваться, если не делал этого ранее.

Далее пользователь может выбрать один из возможных функционалов, предоставленных в меню в верхней части экрана.

Если пользователь не имеет дополнительных ролей в базе данных, то ему будут доступны следующие возможности: пользовательский чат, где клиент сможет общаться с остальными пользователями веб-сервиса, личный кабинет, где можно изменить данные своего аккаунта или сменить аватар, страница с соревнованиями, где можно зарегистрировать свою команду на соревнование, страница команд, где можно создать или вступить в команду (Рисунок Б.1).

Если пользователь имеет права администратора, то ему станет доступна панель администрирования, где он может блокировать/разблокировать аккаунты, создать пользователей, выдавать/забирать привилегии (Рисунок Б.2).

Если пользователь имеет права организатора, то на странице соревнований, появится дополнительный выпадающий список с действиями над ними: создать соревнование, удалить соревнование, отредактировать соревнование. Однако пользователи не смогут сразу увидеть созданное организатором соревнование, так как оно, после создания, отправляется на рассмотрение модератору (Рисунок Б.3).

Если пользователь имеет права модератора, у него появится дополнительная страница со списком соревнований, отправленных для модерации. Он может как одобрить его, в следствии чего оно начнет отображаться у всех, либо отклонить, тогда оно останется на странице у модератора с соответствующей пометкой. Все выше перечисленных варианты

использования отображены на диаграммах вариантов использования (Рисунок Б.4).

На основе предметной области и информации о предоставляемой функциональности была разработана модель бизнес-процесса записи на соревнование (Рисунки 2.1-2.3).

На контекстной диаграмме в качестве входов представлены персональные данные пользователя, список команд и список проверенных соревнований.

В качестве управления представлены закон о персональных данных, политика сайта и закон о достоверности информации.

В качестве механизмов представлены ПО веб-сервиса, пользователь и организатор.

Выходом же является нотификация на почтовый ящик пользователя.

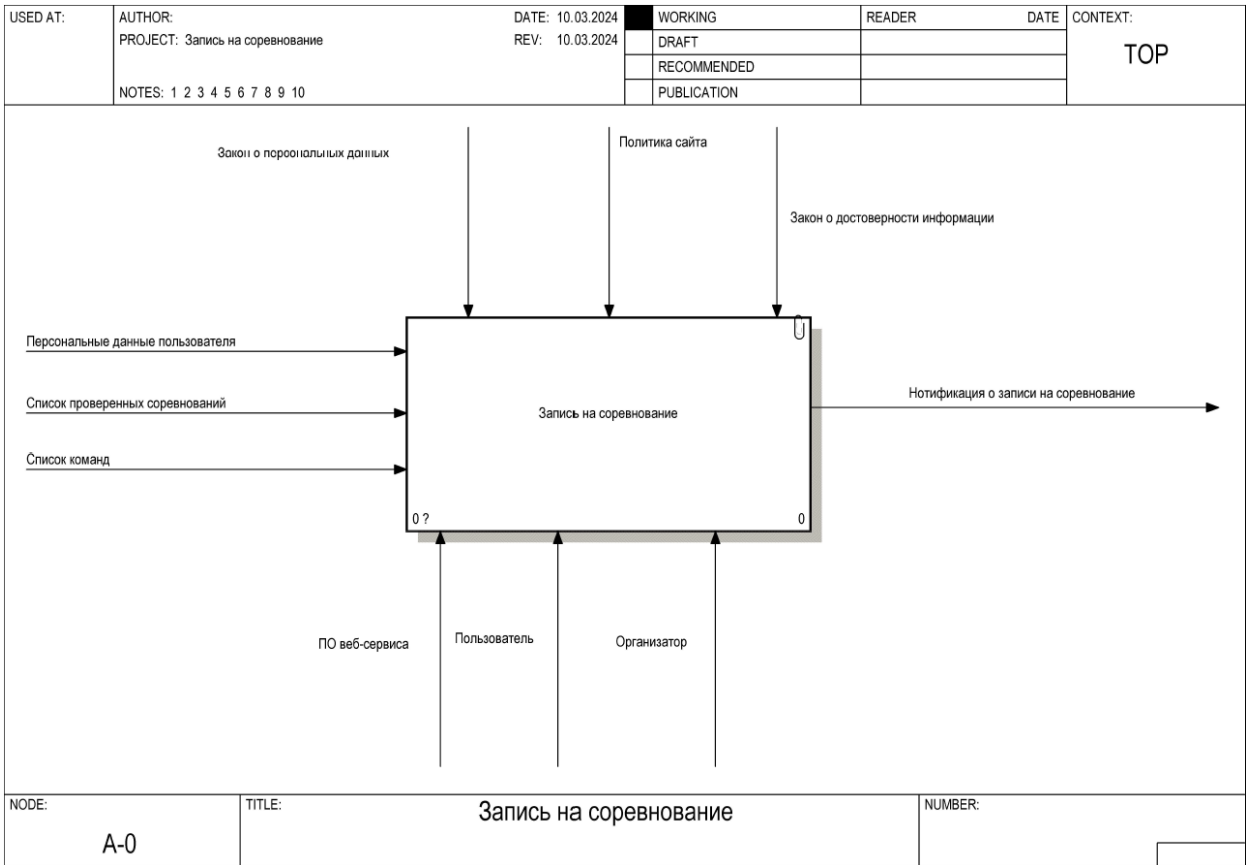


Рисунок 2.1 – Контекстная диаграмма бизнес-процесса «Запись на соревнование» [разработано автором]

На декомпозиции контекстной диаграммы можно видеть три функциональных блока: регистрация пользователя в системе, на вход которому поступают персональные данные пользователя, вступление в команду, на вход которому поступает зарегистрированный пользователь и регистрация на соревнование, на вход которому поступает информация о команде пользователя.

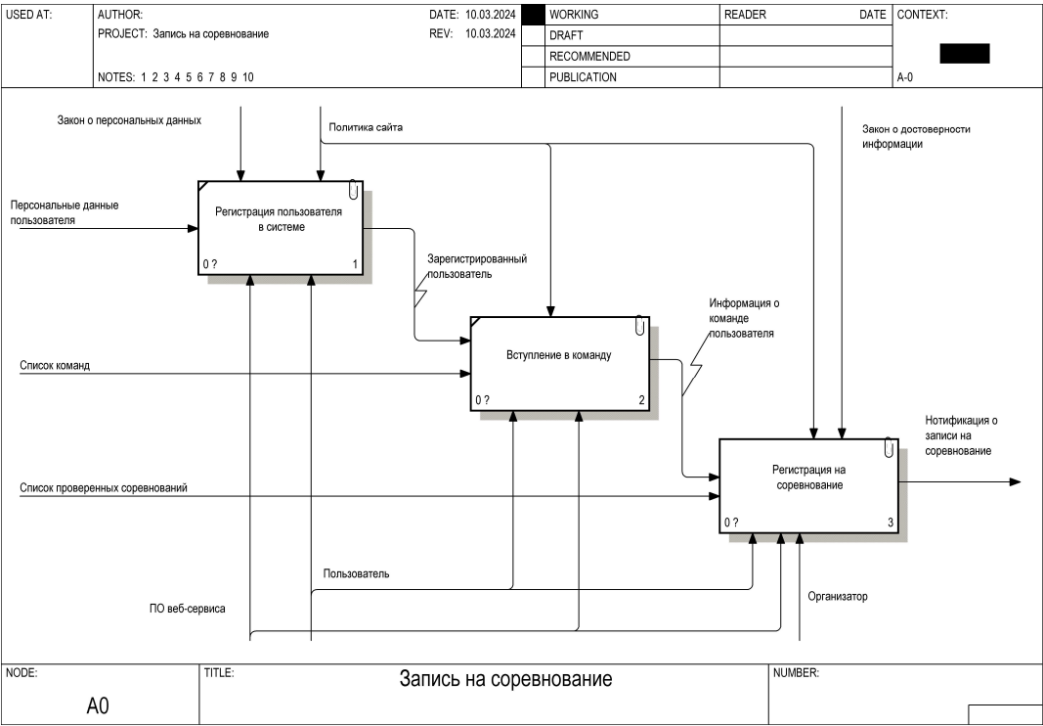


Рисунок 2.2 – Декомпозиция контекстной диаграммы [разработано автором]

Пользователь, желающий записаться на одно из соревнований, начинает с регистрации в системе. После заполнения персональных данных, система проверяет их на корректность и в случае успешной регистрации, пользователю открывается полный функционал сайта.

Затем пользователь переходит на вкладку с командами и вступает в одну из них. Далее пользователь или его команда выбирают интересующее соревнование из списка доступных и регистрируются на него.

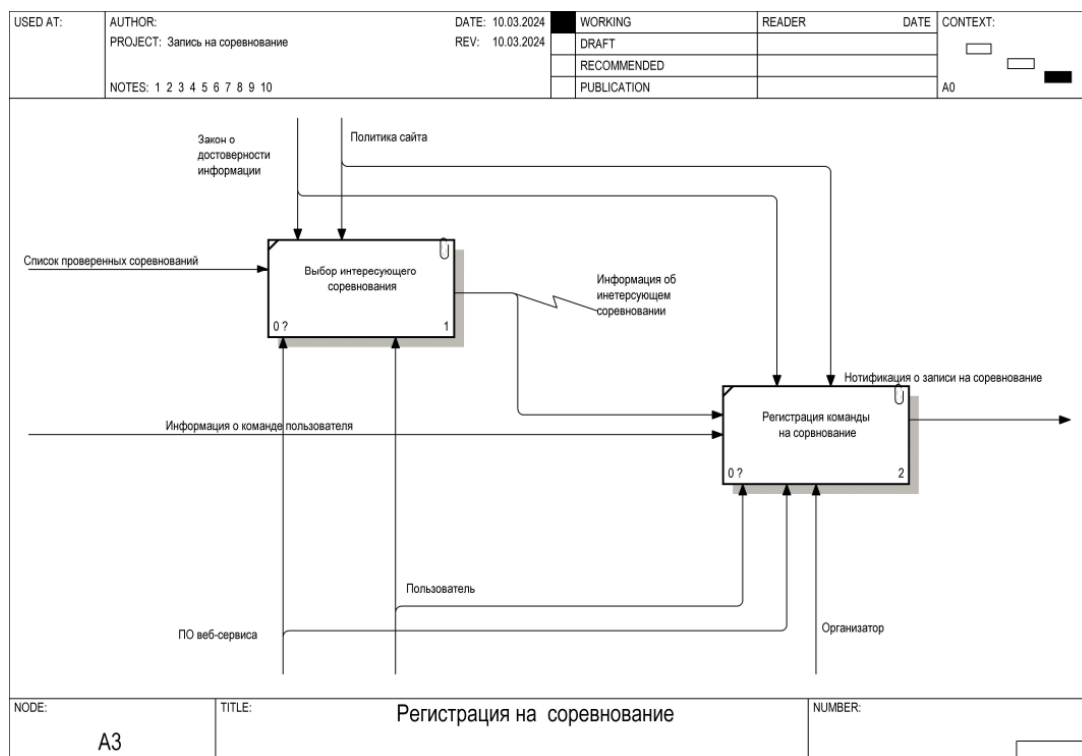


Рисунок 2.3 – Декомпозиция блока «Регистрация на соревнование»
[разработано автором]

На декомпозиции блока «Регистрация на соревнование» представлены два функциональных блока: выбор интересующего соревнования, на вход которому поступает список проверенных соревнований и регистрация команды на соревнование, на вход которому поступает информация об интересующем соревновании.

После завершения процесса регистрации на соревнование пользователь получает уведомление на свой почтовый ящик о подтверждении регистрации. Это уведомление может содержать дополнительную информацию о правилах соревнования, месте и времени его проведения, а также другие важные детали.

Таким образом, бизнес-процесс обеспечивает пользователю удобный и надежный способ записи на соревнование, предоставляя ему необходимую информацию и обеспечивая соответствие всем правилам и требованиям.

2.2 Проектирование адаптированной модели жизненного цикла

В разделе проектирования адаптированной модели жизненного цикла проекта, выбор методологии разработки играет ключевую роль. Одной из

наиболее эффективных и широко используемых методологий в современной разработке программного обеспечения является Scrum [14].

Scrum представляет собой гибкую и итеративную методологию разработки, основанную на принципах коллективной работы, прозрачности и постоянного совершенствования. Его цель состоит в том, чтобы создать ценный продукт с минимальными рисками и максимальной адаптивностью к изменяющимся требованиям рынка.

Спринты (Sprints): Проект разбивается на небольшие временные интервалы, называемые спринтами, обычно длительностью от одной до четырех недель.

Продуктовый бэклог (Product Backlog): Это список всех задач и требований, необходимых для создания продукта.

Спринтовый бэклог (Sprint Backlog): На начало каждого спринта из продуктового бэклога выбираются задачи, которые команда планирует выполнить в течение этого спринта.

Адаптированная модель жизненного цикла представлена на рисунке 2.4.

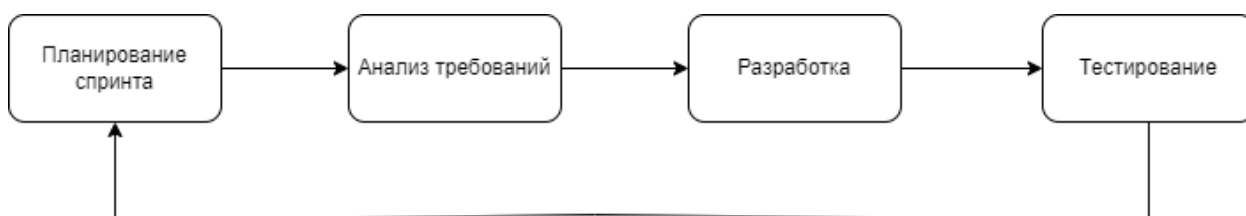


Рисунок 2.4 – Адаптированная модель жизненного цикла

При адаптации модели жизненного цикла проекта, были убраны ежедневные созвоны (Daily Standups) и обзоры спринта (Sprint Reviews), так как разработка проекта ведется одним человеком. Это позволяет сэкономить время на командный анализ требований и позволяет сконцентрироваться на проектировании и разработке заявленного функционала.

2.3 Проектирование базы данных

Главная задача системы – сбор и обработка информации об основных участниках процесса: пользователях, соревнованиях, командах, медиа информации, а также истории сообщений в чатах. При разработке системы

следует учитывать, что она основывается на политике интернет-сайта. Информация о действиях пользователей должна накапливаться и храниться в течение всего существования профиля.

Проектируемая система должна выполнять следующие действия:

- хранить информацию о пользователях и их ролях;
- хранить информацию о соревнованиях;
- хранить информацию о командах;
- хранить медиа информацию пользователя;
- хранить историю сообщений в чатах.

Сущности системы:

- пользователь;
- роль;
- соревнование;
- команда;
- медиа;
- сообщение;
- проверка соревнования.

Логическая диаграмма вместе с сущностями проектировалась в Visual Paradigm [17] (Рисунок 2.5)

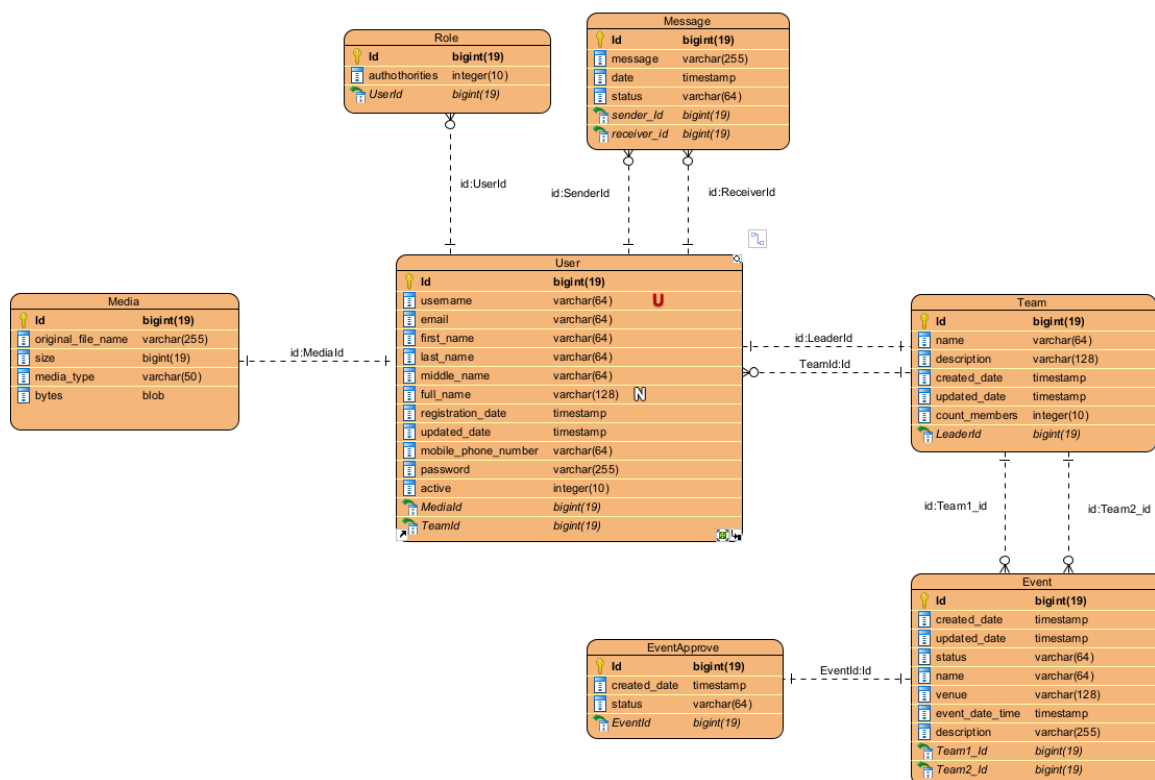


Рисунок 2.5 –Логическая диаграмма базы данных [разработано автором]

Описание каждой из сущности:

- пользователь (Имя пользователя, Почта, Имя, Фамилия, Отчество, Полное имя, Дата регистрации, Дата обновления, Мобильный телефон, Пароль, Активность);
- роль (Название роли);
- соревнование (Дата создания, Дата обновления, Статус соревнования, Название, Место проведения, Дата проведения, Описание соревнования);
- команда (Название команды, Описание команды, Дата создания, Дата обновления, Число участников);
- медиа (Оригинальное название файла, Размер файла, Тип файла, Байты);
- сообщение (Текст сообщения, Дата отправки, Статус сообщения);
- проверка соревнования (Дата создания, Статус проверки).

Следует описать связи между таблицами:

- у одного пользователя может быть несколько ролей. Связь «один-ко-многим»;
- у одного пользователя может быть один медиа файл. Связь «один-к-одному»;
- у одного пользователя может быть много сообщений. Связь «один-ко-многим». Две связи как отправитель и принимающий;
- у одной команды может быть несколько пользователей. Связь «один-ко многим». Также у команды есть лидер. Связь «один-к-одному»;
- у одной команды может быть несколько пользователей. Связь «один-ко многим». Также у команды есть лидер. Связь «один-к-одному»;
- у одного соревнования может быть две команды. Связь «один-ко-многим»;
- у рассмотрения события есть только одно событие. Связь «один-к-одному».

2.4 Проектирование архитектуры клиентской части

Разрабатываемый ресурс должен отвечать требованиям современной веб-разработки:

- предоставлять возможности регистрации и авторизации;
- поддерживать ролевую модель доступа к данным;
- обладать интуитивно понятной навигацией (шапкой сайта с основными разделами, а также выпадающим меню для доступа к дополнительным функциям);
- в разработке должен прослеживаться единый стиль оформления, приятный глазу, но не отвлекающий от основного контента.

На рисунках В.1-В.4 представлены некоторые макеты веб-страниц в планируемой цветовой схеме разрабатываемого приложения.

Клиентская часть – предоставляет интерфейс взаимодействия с веб-приложением. На рисунке 2.6 представлена архитектура клиентской чатси.

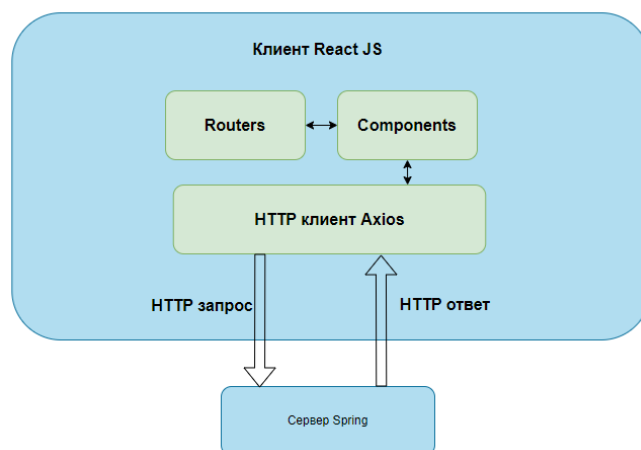


Рисунок 2.6 –Архитектура клиентской части [разработано автором]

Компоненты маршрутизатора в React, такие как BrowserRouter, Switch, Route и Link, предоставляют инфраструктуру для управления навигацией в приложении. Они определяют, какие компоненты отображаются на странице в зависимости от текущего URL-адреса.

Когда пользователь взаимодействует с интерфейсом приложения, например, переходит по ссылке или вводит URL-адрес, компонент маршрутизатора перехватывает это действие и решает, какой компонент нужно отобразить на странице.

В момент активации маршрута (Route), который соответствует текущему URL-адресу, React вызывает соответствующий компонент. Внутри этого компонента может потребоваться загрузить данные с сервера.

В этом случае используется клиентский HTTP-клиент Axios. Он предоставляет удобный интерфейс для создания и отправки HTTP-запросов из JavaScript-кода в React-приложениях.

Например, при загрузке содержимого страницы, компонент маршрутизатора может вызывать функцию Axios для отправки GET-запроса на сервер. В ответ на этот запрос сервер возвращает данные, которые могут быть использованы для отображения на странице.

2.5 Проектирование архитектуры серверной части

На сегодняшний день в промышленной разработке наиболее распространенным подходом к архитектуре ПО считается микросервисный

подход. Он предполагает разделение системы на набор независимых модулей, или микросервисов, вместо единого целого, как в монолитной архитектуре.

Хотя многие ресурсы и эксперты относятся к монолитной архитектуре как к устаревшему подходу, есть ситуации, когда монолит может быть предпочтительным. Например, при использовании каскадной модели жизненного цикла разработки, монолит может быть более удачным выбором, так как микросервисы могут создаваться не всегда эффективно без четкой модульности.

Также существует методика, при которой приложение изначально проектируется как монолитное и поставляется заказчику. Это позволяет создать минимально жизнеспособный продукт (MVP) [18]. После этого, в процессе поддержки и сопровождения, команда разработчиков начинает разбивать монолит на микросервисы постепенно.

С учетом выбранной модели жизненного цикла и оценки имеющихся ресурсов, микросервисная архитектура представляется наиболее подходящим выбором [19] (Рисунок 2.7).

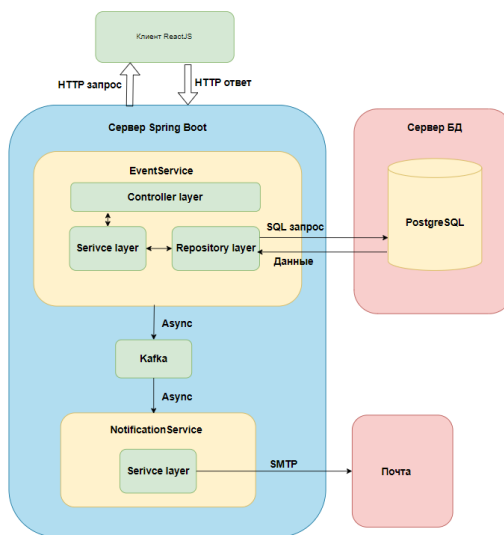


Рисунок 2.7 – Архитектура серверной части [разработано автором]

На данный момент предполагается два микросервиса:

Микросервис соревнований (EventService) – это сервис, который на данный момент отвечает за довольно обширное количество функций, однако может быть разбит в будущем.

Микросервис нотификаций (NotificationService) – это сервис, который отвечает за рассылку нотификаций по различным условиям. По топику кафки в сервис приходят Json объекты, на основе которых происходит логика рассылки нотификаций.

В разрабатываемом приложении для реализации серверной части в качестве паттерна проектирования был выбран паттерн Model-View-Controller.

Model-View-Controller (MVC) — набор архитектурных идей и принципов для построения сложных систем с пользовательским интерфейсом [20]. Имеет определенные различия с паттерном MVP. Данный паттерн описывает простой способ построения структуры приложения, целью которого является отделение бизнес-логики от интерфейса пользователя. Структура MVC представлена на рисунке 2.8.

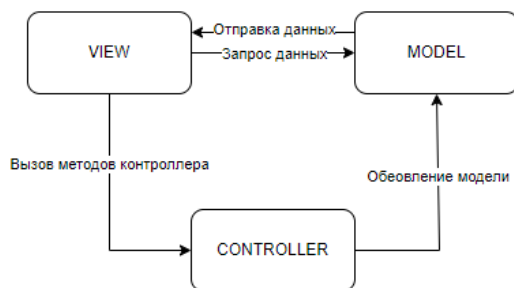


Рисунок 2.8 – Структура MVC

Данный паттерн используется для того, чтобы разделить логические части приложения и создавать их отдельно друг от друга. То есть писать независимые блоки кода, которые можно редактировать, не внося изменений в другие.

Паттерн MVC предлагает множество преимуществ для программиста при разработке приложений, которые включают в себя:

- несколько разработчиков могут работать с тремя слоями (Model, View и Controller) одновременно;
- поскольку компоненты имеют низкую зависимость друг от друга, их легко поддерживать;

- модель может быть повторно использована несколькими представлениями, что обеспечивает возможность повторного использования кода;
- MVC делает приложение более выразительным и простым для понимания;
- масштабируемость и тестирование приложения становится более удобным и легко реализуемым.

Альтернативой MVC является шаблон проектирования MVP (Model-View-Presenter). Он имеет схожую структуру, но с некоторыми отличиями.

MVC был выбран из-за своей широкой распространенности, простоты и понятности. Кроме того, фреймворк Spring предоставляет встроенную поддержку для MVC, что делает его лучшим выбором для разработки серверной части.

2.6 Выводы к разделу 2

В данном разделе разработана архитектура веб-сервиса, включая бизнес-процессы, жизненный цикл проекта, базу данных, серверную и клиентскую части. Бизнес-процессы охватывают регистрацию, управление командами и участие в соревнованиях для различных ролей пользователей. Выбрана методология Scrum и практики DevOps для гибкой и эффективной разработки. Использована микросервисная архитектура и паттерн MVC с поддержкой Spring для повышения гибкости и масштабируемости.

3 Технологический раздел

3.1 Разработка бизнес-логики приложения

Разработка бизнес-логики приложения заключается в прорабатывании основных функций и правил взаимодействия пользователя с системой. В данном случае, она включает аутентификацию, авторизацию пользователей с использованием связки технологий Spring Security и JWT для безопасного хранения приватной информации.

После регистрации/авторизации/аутентификации пользователь получает полный доступ к возможностям системы в зависимости от его ролей в системе.

Обычные пользователи могут управлять своими профилями, изменяя ФИО, электронную почту, пароль и фотографию, могут создавать команды, вступать в них или выходить из них, при этом лидер команды управляет составом участников, также лидер команды может регистрировать команду на выбранное соревнование.

Администраторы имеют возможность просматривать, фильтровать и управлять пользователями.

Модераторы видят созданные события и могут их принимать или отклонять, в то время как организаторы создают и управляют событиями,

Реализован онлайн-чат с общими комнатами и приватными диалогами, позволяющий пользователям отправлять сообщения друг другу.

3.2 Разработка клиент-серверного приложения

Серверная часть веб-приложения реализует паттерн MVC. Логика приложения разделена на отдельные компоненты, такие как: классы-контроллеры, классы-модели, классы-сервисы, классы-репозитории, классы-мапперы, классы-конфигурации, классы-ошибки. После инициализации Spring Boot приложения, были подключены необходимые проекту зависимости в файл pom.xml.

3.2.1 Разработка клиентской части

Клиентская часть приложения разработана с использованием библиотеки React для создания динамических и интерактивных пользовательских интерфейсов. Это веб-приложение предназначено для обеспечения удобного и интуитивно понятного интерфейса пользователя для взаимодействия с бэкэндом и получения информации. Структура клиентской части представлена на рисунке 3.1.

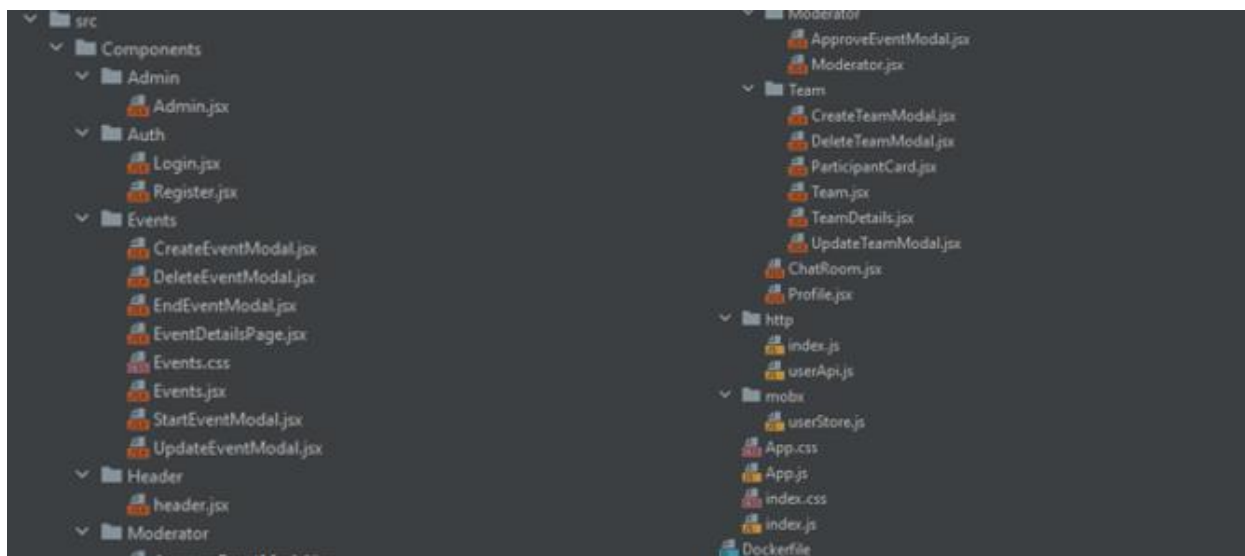


Рисунок 3.1 – Структура клиентского приложения [разработано автором]

На рисунке 3.1 можно видеть директорию с компонентами приложения: Admin, Auth, Events, Header, Moderator, Team, ChatRoom, Profile. А также директорию http, в которой прописаны структуры запросов к серверной части. Немало важную роль играет и UserStore, который отвечает за сохранение состояния авторизованного пользователя, благодаря чему можно не обращаться каждый раз к серверу для сбора данных о нем.

Рассмотрим примеры разработанных экранов приложения (Рисунки Д.1 – Д.7).

Страница логина включает три поля для ввода: никнейм, почта и пароль. Пользовательский чат отображает онлайн-чат с одной общей комнатой и комнатой с самим собой. При авторизации другого пользователя он появится в левом меню чата и станет доступен для отправки сообщений. Профиль позволяет изменить данные аккаунта: ФИО, почта, мобильный телефон, а

также пароль и фотографию. Администратор видит список пользователей с возможностью фильтрации по логину, почте и т.д., а также основные доступные действия. Модератору показываются события, созданные организатором, с возможностью их принятия или отклонения. Список соревнований создается организаторами. Страница команд содержит список команд для вступления или выхода, а также дополнительную информацию о текущем составе команды. Состав команды отображает участников, где создатель команды считается лидером, а остальные участниками.

3.2.2 Разработка серверной части

В начале разработки сервиса соревнований необходимо позаботиться об удобстве сопровождения разработки при помощи CI/CD в GitLab [21]. Для этого необходимо написать файл конфигурации для Continuous Integration/Continuous Deployment (CI/CD) с фазами build и test, что предоставляет дополнительные удобства сопровождения разработки:

- автоматизация процесса сборки и тестирования: Файл конфигурации CI/CD позволяет определить шаги, необходимые для сборки и тестирования вашего проекта автоматически. Это упрощает процесс разработки, поскольку каждое изменение в коде будет автоматически проверяться на соответствие стандартам и прохождение тестов;

- быстрая обратная связь: Запуск автоматической сборки и тестирования по каждому коммиту позволяет получить быструю обратную связь о состоянии проекта. Если какие-то тесты не проходят или сборка не удастся, разработчик может быстро определить проблему и исправить ее, минимизируя время, потраченное на поиск ошибок;

- гарантированное качество: Тестирование является важной частью разработки программного обеспечения, особенно в проектах Java Spring. Наличие автоматических тестов в процессе CI/CD позволяет гарантировать качество кода и предотвращать внесение ошибок и дефектов в основную ветку репозитория;

– стандартизация процесса разработки: Файл конфигурации CI/CD определяет набор шагов, которые выполняются при каждом коммите или pull request. Это помогает стандартизировать процесс сборки, тестирования и развертывания, что делает его более надежным и предсказуемым для всех участников команды.

Следующим шагом необходимо развернуть необходимую инфраструктуру: контейнер с PostgreSQL [22], контейнер с Kafka [23] и ZooKeeper [24], а также контейнер с pgAdmin.

Хорошей практикой является использование инструментов контроля версионности базы данных, особенно в контексте командой разработки, когда несколько человек могут одновременно редактировать структуру базы данных. Для этого используется библиотека liquibase. Эти изменения можно будет откатить также, как и неудачные коммиты в git.

Далее были написаны классы, отвечающие за конфигурацию отдельных компонентов приложения, таких как: Spring Security, Spring Web, Kafka, WebSocket и тд.

Основным инструментом, которое используется для идентификации пользователей, являются токены, генерируемые сервером. Применяется стандарт JWT [25], который предоставляет безопасный и эффективный способ аутентификации пользователей, обеспечивая им уникальные идентификаторы для взаимодействия с нашим приложением.

Был интегрирован этот стандарт, что обеспечило надежную систему аутентификации и безопасный обмен данными между клиентом и сервером. Благодаря этому гарантируется конфиденциальность и целостность информации пользователей, обеспечивая пользователям безопасный и комфортный опыт использования приложения.

Для работы с базой данных и сущностями в Java Spring использовался Hibernate с JPA (Java Persistence API), так как они предоставляют удобные инструменты для работы с объектно-реляционным отображением данных. Были написаны классы, отображающие модели данных: User, Event, Team,

EventApprove, Media, Message, а также необходимые к ним DTO (Data Transfer Object) [26], в частности их реализации для основных CRUD операций (Create...DTO, PATCH...DTO и тд) (Рисунок 3.2). В листинге Г.1 можно видеть пример реализации одной из моделей – User.

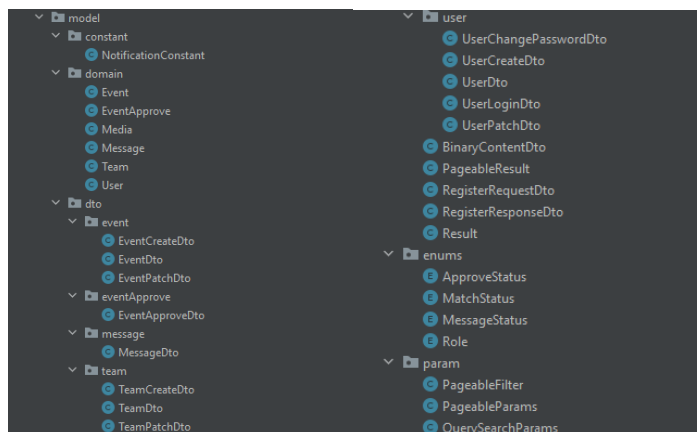


Рисунок 3.2 – Классы моделей и DTO приложения [разработано автором]

Вся основная бизнес логика приложения находится в классах сервисах (Рисунок 3.3).

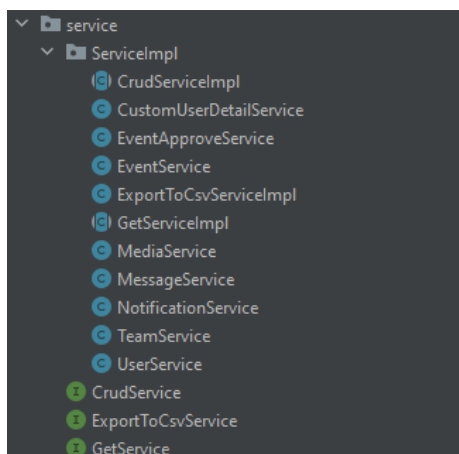


Рисунок 3.3 – Классы сервисы приложения [разработано автором]

Класс CrudServiceImpl является абстрактным и предоставляет реализацию методов создания, обновления и удаления записей. Он расширяет класс GetServiceImpl, который в свою очередь реализует методы получения данных.

Рассмотрим основные компоненты этого кода:

Обобщенные типы:

- E - тип сущности;

- CreateDto - тип DTO (Data Transfer Object) для создания сущности;
- PatchDto - тип DTO для обновления сущности;
- Dto - общий тип DTO;
- Id - тип идентификатора сущности;
- R - тип репозитория, расширяющий EntityRepository.

Конструкторы:

- CrudServiceImpl принимает репозиторий и маппер и сохраняет их в поля;
- GetServiceImpl также принимает репозиторий и маппер.

Методы:

- create(CreateDto createDto): Создает новую сущность на основе переданного DTO и сохраняет ее в репозиторий. Затем преобразует сущность в DTO и возвращает;
- update(PatchDto patchDto, Id id): Обновляет существующую сущность с указанным идентификатором на основе данных из DTO обновления. Сущность сохраняется в репозитории, затем преобразуется в DTO и возвращается;
- delete(Id id): Удаляет сущность с указанным идентификатором из репозитория.

Пример кода, который предоставляет основу для реализации служб, работающих с сущностями в приложении, обеспечивая стандартные операции CRUD и операции получения данных находится в листингах Г.2-Г.3.

Для взаимодействия с базой данных из кода были написаны классы репозитории, который наследуются от базового интерфейса Spring Data JPA – JpaRepository.

Для автоматического сопоставления объектов была использована библиотека MapStruct. Она позволяет генерировать Java-код для сопоставления между различными классами, что упрощает код и избавляет от рутинных задач ручного сопоставления, например сопоставление между DTO

и сущностями JPA [27]. Пример одного из мапперов (UserMapper) приведен в листинге Г.4.

Одним из заключительных этапов разработки является написание контроллеров, предоставляющих конкретные эндпоинты, по которым можно обращаться для получения нужной информации к серверной части приложения. Пример одного из таких контроллеров представлен в листинге Г.5.

Сервис нотификаций разработан для получения сообщений из Apache Kafka и отправки их на электронную почту. Он представляет собой часть микросервисной архитектуры, предназначенной для обеспечения моментальной связи с пользователями при поступлении определенных событий.

Структура данного сервиса схожа со структурой сервиса соревнований. Были написаны классы конфигурации, в которых указывается чувствительные данные почты, с которой будет происходить отправка сообщений.

А также классы сервисы, которые обрабатывают сообщения, вычитанные из топика кафки (Рисунок 3.4).

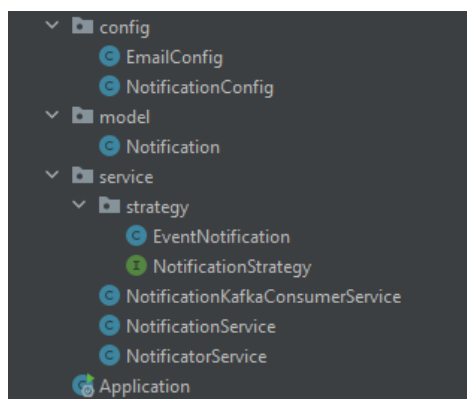


Рисунок 3.4 – Структура сервиса нотификаций [разработано автором]

3.3 Тестирование приложения на соответствие требованиям

Написание как юнит-тестов, так и интеграционных тестов является критически важным этапом в разработке программного обеспечения. Юнит-тесты [28] проверяют отдельные компоненты кода на корректность их работы,

что помогает выявить и исправить ошибки на ранних этапах разработки. Они также обеспечивают документацию к коду и уменьшают риск возникновения ошибок при изменениях в коде.

С другой стороны, интеграционные тесты [29] проверяют взаимодействие между различными компонентами системы и их корректность в совокупности. Эти тесты обнаруживают ошибки, которые могут возникнуть при интеграции компонентов, а также помогают проверить работу приложения в реальной среде.

Вместе юнит-тесты и интеграционные тесты обеспечивают полное покрытие кода тестами, что повышает надежность приложения, улучшает его качество и ускоряет процесс разработки. Эти тесты помогают предотвратить появление серьезных проблем и ошибок в продукте, обеспечивая его стабильную и безопасную работу (Рисунок 3.5).

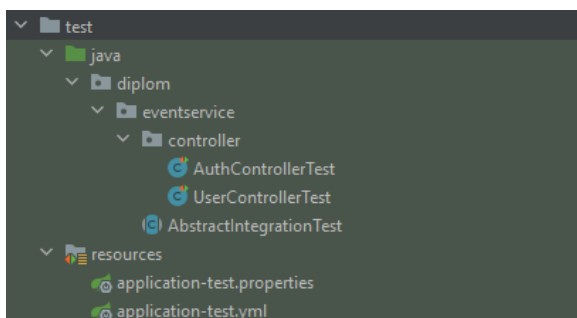


Рисунок 3.5 – Тесты серверной части приложения [разработано автором]

Корректность прохождения тестов можно увидеть в пайплайне репозитория на гитлабе. Все зеленые джобы сигнализируют о том, что все написанные тесты завершились успешно (Рисунок 3.6).

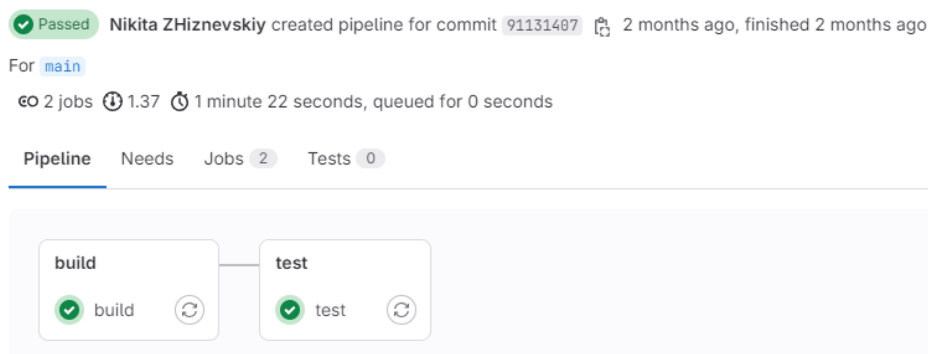


Рисунок 3.6 – Успешно пройденные пайплайны [разработано автором]

Для обеспечения надежности и корректности работы серверной части приложения были разработаны и выполнены тестовые сценарии, направленные на проверку ключевых функций системы. Это включает в себя тестирование процессов получения токена аутентификации, а также взаимодействия с базой данных через различные API-запросы.

На рисунке 3.7 представлен запрос на получения токена аутентификации к приложению, на котором видно, что в ответе сервера был получен JWT токен и данные, зарегистрировавшего пользователя.

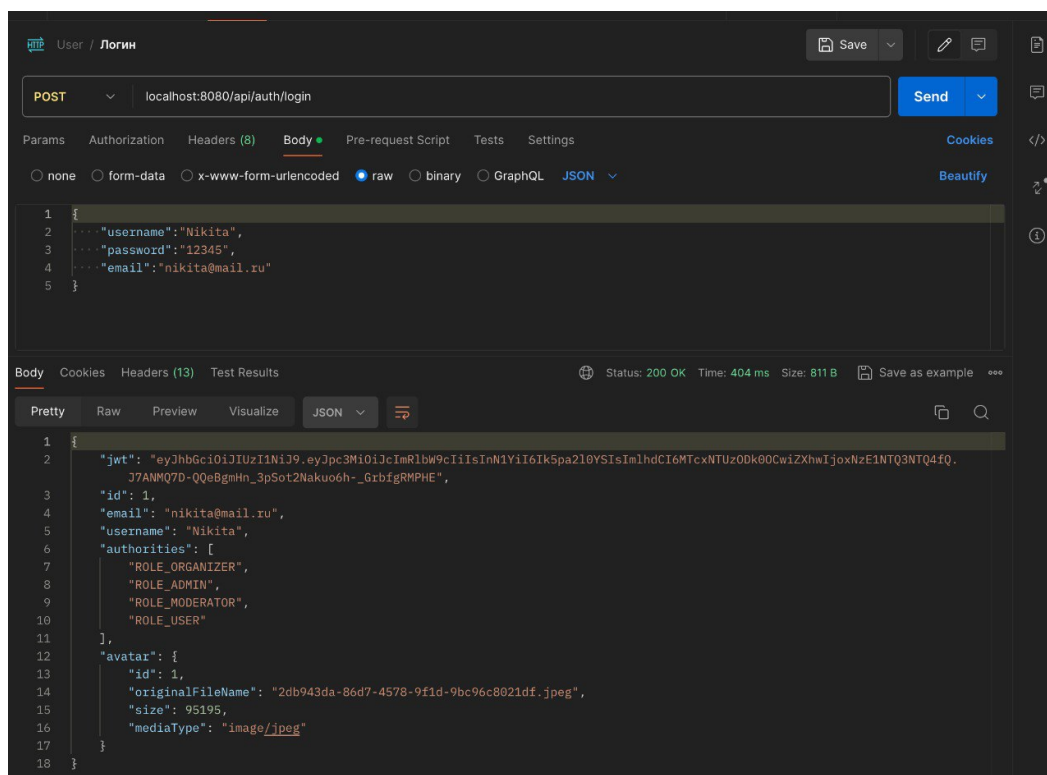


Рисунок 3.7 – Тест логина в системе [разработано автором]

На рисунке 3.8 представлен тест получения списка соревнований, на котором видно, что на запрос получения списка соревнований сервер отдал все существующие в системе соревнования с пагинацией и фильтрацией.

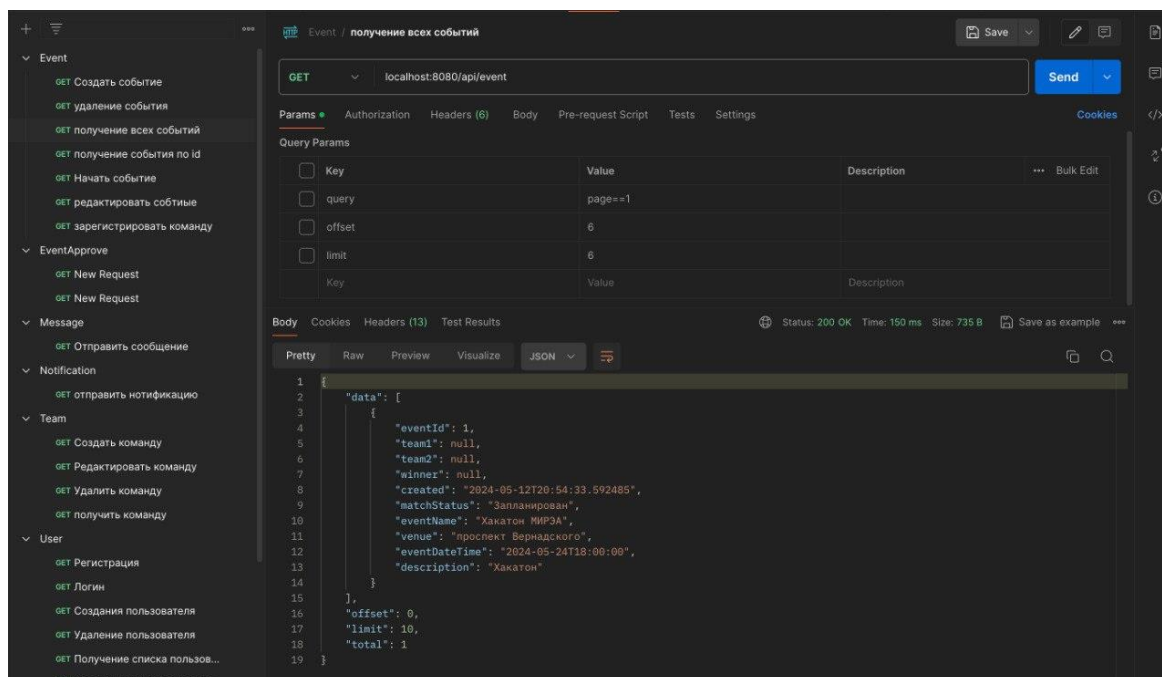


Рисунок 3.8 – Тест получения списка соревнований [разработано автором]

Результаты тестирования, выполненные с помощью Postman, были зафиксированы и проанализированы. Проверка показала, что серверная часть приложения отвечает на запросы в соответствии с ожиданиями, обрабатывая их корректно и возвращая соответствующие ответы. Это подтверждает готовность серверной части к дальнейшей интеграции с клиентской частью и последующему внедрению в эксплуатацию.

3.4 Расчет вычислительной и ёмкостной сложностей

Для клиент-серверного приложения потребуются следующие вычислительные ресурсы и требования к подключению.

Сервер должен иметь минимум 4 ядра процессора с тактовой частотой не менее 2.0 ГГц для обработки запросов и выполнения бизнес-логики. Необходимо наличие не менее 8 ГБ оперативной памяти (RAM) для эффективной работы базы данных и приложения. Для хранения данных рекомендуется использовать SSD диск емкостью от 100 ГБ. Также необходимо обеспечить подключение к интернету с пропускной способностью не менее 100 Мбит/с для быстрого обмена данными с клиентами.

Эти требования обеспечат надежную и эффективную работу приложения, быструю обработку запросов и комфортное взаимодействие

пользователей с системой. В таблице 3.1 представлены показатели вычислительной и ёмкостной сложности методов приложения.

Таблица 3.1 – Вычислительная и ёмкостная сложности методов приложения [разработано автором]

Метод	Операции	Сложность
create(EventCreateDto eventCreateDto)	Преобразование DTO в сущность, сохранение в БД, создание и сохранение одобрения, отправка уведомлений	$O(M)$
checkThatTeamsExist(Set<Long> teamsId)	Проверка существования команд и пользователей в командах в БД	$O(T * \log(T))$
update(EventPatchDto eventPatchDto, Long aLong)	Вызов родительского метода	$O(1)$
delete(Long aLong)	Вызов родительского метода	$O(1)$
get(QuerySearchParams params)	Поиск событий с использованием RSQL, фильтрация событий, сортировка	$O(N * \log(N))$
getById(Long aLong)	Поиск по идентификатору	$O(1)$
startEvent(Long id)	Поиск и обновление события	$O(1)$
endEvent(Long id, Long teamId)	Поиск события и команды, обновление события	$O(1)$

$O(1)$: Постоянная сложность – Операции, выполняемые за фиксированное время благодаря индексам и оптимизированным запросам в базе данных.

$O(M)$: Линейная сложность по количеству модераторов – Зависящая от числа модераторов операция.

$O(T * \log(T))$: Лог-линейная сложность – Проверка существования команд и пользователей в командах, зависящая от числа команд.

$O(N * \log(N))$: Лог-линейная сложность – Поиск и фильтрация событий с сортировкой, где N – количество событий. Такие операции, как сортировка и сложные фильтрации, увеличивают сложность.

$O(\log(T))$: Логарифмическая сложность – Поиск команды по имени в отсортированном наборе данных (индексированные данные).

Таким образом, наиболее сложные с точки зрения вычислительной сложности методы — это те, которые выполняют сортировку, сложные фильтрации.

3.5 Выводы к разделу 3

По итогам третьего раздела были разработаны два микросервиса: соревнований и уведомлений. Были представлены листинги и скриншоты разрабатываемого веб-сервиса. Также, было проведено тестирование веб-сервиса путем запуска модульных и интеграционных тестов и прогона процессов в Postman.

4 Экономический раздел

4.1 Организация и планирование работ

Основные участники проекта:

1. Руководитель (руководитель выпускной квалификационной работы, кандидат технических наук, доцент кафедры ИиППО РТУ МИРЭА).
2. Консультант (консультант по экономической части ВКР, кандидат экономических наук, доцент кафедры РТУ МИРЭА).
3. Разработчик проекта (студент 4-го курса, группа ИКБО-16-20).

На проект отводится 70 рабочих дней. Этапы проекта представлены в таблице 4.1.

Таблица 4.1 – Календарный план выполнения проекта [разработано автором]

Этап	Дата начала	Дата окончания	Количество рабочих дней	Исполнители
1 Техническое задание				
1.1 Обследование объекта и обоснование необходимости создания решения	09.02.2024	09.02.2024	1	Разработчик проекта
1.2 Изучение объекта и разработка вариантов реализации, утверждение выбранного варианта	10.02.2024	10.02.2024	1	Руководитель ВКР Разработчик проекта
1.3 Разработка и утверждение технического задания	12.02.2024	13.02.2024	2	Руководитель ВКР Разработчик проекта
2 Эскизный проект				
2.1 Анализ предметной области	14.02.2024	15.02.2024	2	Разработчик проекта
2.2 Анализ конкурентных решений	16.02.2024	17.02.2024	2	Разработчик проекта
2.3 Анализ и выбор технологий для разработки	19.02.2024	20.02.2024	2	Разработчик проекта

Продолжение таблицы 4.1

Этап	Дата начала	Дата окончания	Количество рабочих дней	Исполнители
2.4 Разработка общего описания алгоритма функциональных возможностей программы	21.02.2024	24.02.2024	3	Руководитель ВКР Разработчик проекта
2.5 Постановка задачи	26.02.2024	29.02.2024	4	Руководитель ВКР Разработчик проекта
3 Технический проект				
3.1 Разработка функциональной схемы программы	01.03.2024	05.03.2024	4	Руководитель ВКР Разработчик проекта
3.2 Разработка структуры программы	06.03.2024	12.03.2024	5	Руководитель ВКР Разработчик проекта
3.3 Разработка логической схемы базы данных	13.03.2024	16.03.2024	4	Руководитель ВКР Разработчик проекта
4 Рабочий проект				
4.1 Разработка программы	18.03.2024	09.04.2024	20	Разработчик проекта
4.2 Разработка архитектуры решения	10.04.2024	15.04.2024	5	Разработчик проекта
4.3 Тестирование программы	16.04.2024	18.04.2024	3	Разработчик проекта
4.4 Доработка программного кода	19.04.2024	22.04.2024	3	Разработчик проекта
4.5 Подготовка технической документации на программный продукт	23.04.2024	25.04.2024	3	Разработчик проекта
4.6 Расчет затрат на проведение работ	26.04.2024	26.04.2024	1	Консультант по экономической части Разработчик проекта
4.7 Сдача готового проекта и внедрение	27.04.2024	03.05.2024	5	Руководитель ВКР Консультант по экономической части Разработчик проекта

Календарный план-график, разработанный в соответствии данными из таблицы 4.1, был построен в виде диаграммы Ганта (Рисунок Е.1). Диаграмма Ганта широко используется для визуализации хода выполнения задач, планирования ресурсов, графика рабочего времени и других данных, которые представляются набором временных интервалов [30, 31, 32]. Она необходима

для соблюдения сроков выполнения работ, анализа хода проектирования и разработки ИТ-решения.

4.2 Расчёт затрат на проведение работ

4.2.1 Расчет основной заработной платы

Для определения основной заработной платы необходимо проанализировать уровень заработных плат специалистов, привлеченных к выполнению проекта. Информация об оплате труда руководителя ВКР и консультанта по экономической части была получена с официального сайта РТУ МИРЭА и составляет 138 955 рублей [33]. Для удобства расчетов данное значение было округлено до 140 000 рублей. Следовательно, необходимо проанализировать только размер оплаты труда разработчика проекта (Таблица 4.2).

Таблица 4.2 – Результаты анализа размера оплаты труда [разработано автором]

Исполнитель (должность)	Ряд значений	Источник данных	μ и Md	Квартили
Разработчик проекта	60 000	https://hh.ru/vacancy/97824754?hhtmFrom=vacancy_search_list	84 000 80 000	70 000 80 000 100 000
	70 000	https://hh.ru/vacancy/96629164?query=Junior+java&hhtmFrom=vacancy_search_list		
	80 000	https://hh.ru/vacancy/97824754?query=Junior+java&hhtmFrom=vacancy_search_list		
	90 000	https://hh.ru/vacancy/95997045?query=Junior+java&hhtmFrom=vacancy_search_list		
	120 000	https://hh.ru/vacancy/95881434?query=Junior+java&hhtmFrom=vacancy_search_list		

Был сформирован упорядоченный числовой ряд на основе данных из открытых источников о заработных платах разработчиков: 60000, 70000, 80000, 90000, 150000. В столбце 4 указаны средняя арифметическая и медиана выборки.

μ – средняя арифметическая, рассчитывается как сумма всех имеющихся в выборке обработанных зарплатных значений, деленная на количество источников, предоставивших данные по конкретной позиции (1):

$$\mu = \frac{60\,000 + 70\,000 + 80\,000 + 90\,000 + 120\,000}{5} = 84\,000 \text{ руб.} \quad (1)$$

Md (медиана) – число, находящееся ровно в середине числового ряда. В данной выборке медианой является число 80 000, что меньше среднего арифметического, значит, можно утверждать, что компании платят заработную плату данной вакансии меньше.

В последнем столбце таблицы 4.2 рассчитаны квартили. Первый квартиль (Q_1) определяется как среднее арифметическое между наименьшим числом и медианой набора данных (2). Второй квартиль (Q_2) является медианой набора данных (3). Третий квартиль (Q_3) представляет собой среднее арифметическое между медианой и наибольшим значением набора данных (4).

$$Q_1 = \frac{60\,000 + 80\,000}{2} = 70\,000 \text{ руб.} \quad (2)$$

$$Q_2 = Md = 80\,000 \text{ руб.} \quad (3)$$

$$Q_3 = \frac{80\,000 + 120\,000}{2} = 100\,000 \text{ руб.} \quad (4)$$

Нижний квартиль свидетельствует о том, что 25% компаний выплачивают заработную плату ниже 70 000 рублей. Такая сумма может быть установлена на испытательный срок. Второй квартиль (50%) – это средняя заработная плата на рынке труда. Третий квартиль показывает, что 75% компаний выплачивают заработную плату ниже 100 000 рублей, а 25% – выше этой отметки. Он ориентирует работодателя на установление конкурентоспособной заработной платы для привлечения ценных работников.

Согласно производственному календарю на 2024 год, годовой фонд рабочего времени при шестидневной рабочей неделе составит 300 рабочих дней [34]. Следовательно, дневная тарифная ставка (ТС) для месячного оклада (ОК) участников проекта рассчитывается по формуле (5):

$$TC = \frac{OK \times 12}{НРВ} = \frac{OK \times 12}{300} \text{ руб./д.} \quad (5)$$

где ОК - месячный оклад участника проекта,

12 - число месяцев в году,

НРВ - годовой фонд рабочего времени при сорокачасовой рабочей неделе.

Месячный оклад руководителя ВКР и консультанта по экономической части составляет (5):

$$TC = \frac{OK \times 12}{НРВ} = \frac{140\,000 \times 12}{300} = 5\,600 \text{ руб./д.} \quad (5)$$

Месячный оклад разработчика составляет (6):

$$TC = \frac{OK \times 12}{НРВ} = \frac{80\,000 \times 12}{300} = 3\,200 \text{ руб./д.} \quad (6)$$

В Таблице 4.3 приведен расчет заработной платы участников проекта. Количество рабочих дней для каждой роли определяется на основе таблицы 4.1 с учетом участия в выполнении работ.

Таблица 4.3 – Расчет основной заработной платы [разработано автором]

Наименование этапа	Исполнитель (должность)	Мес. оклад (руб.)	Оплата за день (руб.)	Количество рабочих дней	Оплата за этап (руб.)
1 Техническое задание	Руководитель ВКР	140 000	5 600	3	16 800
	Разработчик проекта	80 000	3 200	4	12 800
2 Эскизный проект	Руководитель ВКР	140 000	5 600	7	39 200
	Разработчик проекта	80 000	3 200	13	41 600
3 Технический проект	Руководитель ВКР	140 000	5 600	13	45 760
	Разработчик проекта	80 000	3 200	13	72 800
4 Рабочий проект	Руководитель ВКР	140 000	5 600	5	28 000
	Консультант по экономической части	140 000	5 600	1	5 600
	Разработчик проекта	80 000	3 200	30	96 000
Итого:					358 560

Таким образом, заработная плата участников проекта составляет 358 560 рублей

4.2.2 Вычисление дополнительной заработной платы

Расходы дополнительной заработной платы (ДЗП) составляют 20% от суммы основной заработной платы (7). Также, в процессе определения сметы

затрат вводится понятие «фонд оплаты труда», представляющее собой сумму основной (ОЗП) и ДЗП (8).

$$\text{ДЗП} = 0.2 \times 358\,560 = 71\,712 \text{ руб} \quad (7)$$

$$\text{ФОТ} = \text{ОЗП} + \text{ДЗП} = 358\,560 + 71\,712 = 430\,272 \text{ руб.} \quad (8)$$

4.2.3 Страховые взносы

В соответствии со статьей 425 НК РФ в 2024 году для лиц, которые производят выплаты и вознаграждения физическим лицам, установлен Единый тариф страховых взносов 30%. Также, в расчете необходимо учесть ставку взносов на травматизм [35, 36, 37, 38]. Для РТУ МИРЭА данная ставка составляет 0,2%. Таким образом, страховые взносы (СВ) во внебюджетные фонды составят 30,2% от фонда оплаты труда (ФОТ) (9):

$$\text{СВ} = 430\,272 \times 0.302 = 129\,942,144 \text{ руб} \quad (9)$$

4.2.4 Материалы, покупные изделия и полуфабрикаты

В таблице 4.4 представлен расчет затрат на покупку материалов, покупных изделий и полуфабрикатов.

Таблица 4.4 – Расчет затрат на материалы [разработано автором]

№ пп	Наименование материалов	Единицы измерения	Количество	Цена за единицу (руб)	Стоимость (руб)
1	Флешка 4Гб	шт.	1	400	400
2	Бумага А4	пачка	1	370	370
3	Картридж для принтера	шт.	1	2500	2 500
4	Канцелярский набор	шт.	1	350	350
Итого материалов					3 620
Транспортно-заготовительные расходы					418
Итого					4 038

Таким образом, затраты на материалы составляют 4 038 рублей.

Расходы на комплектующие изделия и изготовление программно-аппаратного комплекса не предусмотрены.

4.2.5 Расходы на приобретение компьютерной техники и программного обеспечения

Расходы на приобретение компьютерной техники и программного обеспечения отсутствуют.

4.2.6 Амортизационные отчисления

Амортизационные отчисления – отчисления части стоимости основных фондов для возмещения их износа. Их размер определяется на основе установленных норм и балансовой стоимости основных фондов, на которые начисляется амортизация. Для определения затрат по проекту был использован линейный способ вычисления амортизационных отчислений. К амортизационным отчислениям во время разработки проекта будет относиться сумма используемого во время разработки проекта оборудования к отношению времени использованного данного оборудования (10):

$$A = \frac{C}{T} \quad (10)$$

где А - месячная сумма амортизационных отчислений,

С - первоначальная стоимость объекта,

Т – срок полезного использования в месяцах.

В таблице 4.5 представлены расчеты амортизационных отчислений по использованному во время разработки проекта оборудованию.

Таблица 4.5 – Амортизационные отчисления [разработано автором]

Наименование оборудования	Первоначальная стоимость объекта, руб.	Срок полезного использования, мес.	Месячная сумма амортизационных отчислений	Период эксплуатации в месяцах	Сумма, руб.
Ноутбук	120 000	36	3 333,33	3	9 999,99
Монитор	20 500	24	854,16	3	2 562,48
Компьютер	140 000	36	3 888,88	3	11 666,64
Принтер	20 500	36	569,44	3	1 708,32
Клавиатура	5 000	12	416,66	3	1 249,98
Мышка	3 300	12	275	3	825
Итого:					28 012,41

Таким образом, сумма амортизационных отчислений составляет 28 012,41 рублей.

4.2.7 Эксплуатационные расходы, связанные с использованием компьютерной техники

В рамках проектирования и разработки проекта под эксплуатационными расходами понимаются затраты на электроэнергию. Компьютер и монитор используются на протяжении всей работы над проектом, согласно таблице 4.1 это время составляет 560 часов (70 дней по 8 часов в день на одну единицу техники). Ноутбук эксплуатируется параллельно с компьютером, однако время его использования в 2 раза меньше и составляет 280 часов. Общее время печати документов на принтере составляет 2 часа. Каждая из упомянутой техники обладает своим энергопотреблением. Стоимость 1 кВт электроэнергии в Москве равна 6,43 рублей. [39]. Расчет эксплуатационных расходов представлен в таблице 4.6.

Таблица 4.6 – Эксплуатационные расходы [разработано автором]

Наименование техники	Мощность, кВт	Время использования, ч.	Стоимость 1 кВт, руб.	Расходы, руб.
Ноутбук	0,65	280	6,43	1170,26
Компьютер	0,65	560		2340,52
Монитор	0,035	560		126,3
Принтер	0,25	2		3,22
Итого:				3640,3

Таким образом, траты на эксплуатационные расходы составляют 3640,3 рублей.

4.2.8 Накладные расходы

Сумма накладных расходов определяется процентом от суммы основной заработной платы (ОЗП) и в контексте данного проекта берётся за 100% (11):

$$НР = ОЗП \times 100\% = 358\,560 \times 1,0 = 358\,560 \text{ руб.} \quad (11)$$

4.2.9 Прочие расходы

Прочие расходы отсутствуют.

4.2.10 Калькуляция затрат по проекту

Для определения суммы затрат по всем пунктам была построена таблица общего объёма финансирования проекта (Таблица 4.7).

Таблица 4.7 – Общий объем финансирования проекта [разработано автором]

№	Номенклатура статей расходов	Расходы (руб.)	Доля расходов, %
1	Основная заработная плата	358 560,00	37,5
2	Дополнительная заработная плата	71 712,00	7,5
3	Страховые взносы	129 942,144	13,6
4	Материалы, покупные изделия и полуфабрикаты	4 038,00	0,52
5	Расходы на приобретение компьютерной техники и программного обеспечения	-	-
6	Амортизационные отчисления	28 012,41	3
7	Эксплуатационные расходы, связанные с использованием компьютерной техники	3640,3	0,38
8	Накладные расходы	358 560,00	37,5
9	Прочие	-	-
Итого:		954 464,854	100

Таким образом, общий объём финансирования проекта составил 954 464,854 рублей. Для визуализации долевого состава затрат на финансирование проекта была построена круговая диаграмма (Рисунок 4.1).



Рисунок 4.1 – Круговая диаграмма долевого состава затрат [разработано автором]

4.3 Выводы к разделу 4

Разработка данного продукта обосновывается экономически благодаря сокращению операционных издержек, связанных с организацией соревнований, управлением данными и коммуникацией в спортивном сообществе.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы проведено исследование существующих веб-приложений, используемых для создания, управления и отслеживания спортивных соревнований между искусственным интеллектом; выбраны средства разработки для разрабатываемого веб-приложения. Сформулированы задачи проектирования и реализации веб-приложения.

Был проведён анализ предметной области, по результату которого была подтверждена актуальность изучаемой темы в связи с тем, что искусственный интеллект активно внедряется во все сферы жизни, и спорт не стал исключением.

Был проведён анализ конкурентных решений для выявления их недостатков и формирования требований к разрабатываемому веб-приложению. Были определены пользовательские требования к веб-приложению, которые являются обязательными.

Была спроектирована функциональная схема веб-приложения в нотации IDEF0, была спроектирована архитектура веб-приложения в целом, а также его клиентской и серверной части по отдельности. Также была разработана логическая схема базы данных.

Результаты ВКР обсуждались на международной научной конференции «ИТ-Стандарт» 2024. По результатам конференции была подготовлена и принята к публикации статья на тему «Технологии искусственного интеллекта как важный фактор развития интеллектуального спорта в условиях цифровой трансформации экономики».

Разработка веб-приложения была осуществлена с учетом всех предыдущих процессов, включая исследование предметной области, определение требований и процесс проектирования. В результате был реализован основной функционал клиентской и серверной частей веб-приложения и налажено взаимодействие между ними. Было проведено сквозное тестирование, также серверная часть была протестирована с

помощью юнит и интеграционных тестов. Тестирование показало, что веб-приложение полностью выполняет все поставленные задачи.

В ходе выполнения экономической части были разработаны ключевые разделы бизнес-плана проекта, изучены аспекты организации и планирования работ, а также установлена договорная стоимость, все это оформлено в соответствии с методическими указаниями.

During the final qualifying work, a study was conducted of existing web applications used to create, manage and track sports competitions between artificial intelligence; development tools have been selected for the web application being developed.

An analysis of the subject area was carried out, as a result of which the relevance of the topic under study was confirmed due to the fact that artificial intelligence is actively being introduced into all spheres of life, including sports.

An analysis of competitive solutions was carried out to identify their shortcomings and formulate requirements for the web application being developed. User requirements for the web application have been identified and are mandatory.

A functional diagram of the web application was designed in IDEF0 notation, the architecture of the web application as a whole, as well as its client and server parts separately, was designed. A logical database design was also developed.

The development of the web application was carried out by taking into account all the previous processes, including domain research, requirements definition and design process. As a result, the main functionality of the client and server parts of the web application was implemented and interaction between them was established.

During the implementation of the economic part, key sections of the project business plan were developed, aspects of the organization and planning of work were studied, and the contractual cost was established, all of this was drawn up in accordance with the methodological instructions.

СПИСОК ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. СИБИД. Отчёт о научно-исследовательской работе. Структура и правила оформления. (Дата последнего обращения: 13.03.2024).
2. ГОСТ Р 7.0.100-2018. СИБИД. Библиографическое описание. Общие требования и правила составления (Дата последнего обращения: 13.03.2024).
3. СМКО МИРЭА 7.5.1/03.П.30-19. Порядок проведения государственной итоговой аттестации по образовательным программам высшего образования – программам бакалавриата, программам специалитета, и программам магистратуры (Дата последнего обращения: 13.03.2024);
4. СМКО МИРЭА 7.5.1/03.П.67-19. Положение о выпускной квалификационной работе студентов, обучающихся по образовательным программам подготовки бакалавров (Дата последнего обращения: 13.03.2024).
5. ФГОС ВО 3++. Федеральный государственный образовательный стандарт высшего образования. (Дата последнего обращения: 13.03.2024).
6. Яндекс афиша [Электронный ресурс]: режим доступа: <https://afisha.yandex.ru/moscow>, свободный (Дата последнего обращения: 13.03.2024).
7. Мос. ру афиша [Электронный ресурс]: режим доступа: <https://www.mos.ru/afisha/>, свободный (Дата последнего обращения: 13.03.2024). – Текст: электронный.
8. React [Электронный ресурс]: режим доступа: <https://ru.legacy.reactjs.org/>, свободный (Дата последнего обращения: 13.03.2024).
9. Vue [Электронный ресурс]: режим доступа: <https://ru.vuejs.org/>, свободный (Дата последнего обращения: 13.03.2024).
10. Википедия Java [Электронный ресурс]: режим доступа: <https://ru.wikipedia.org/wiki/Java>, свободный (Дата последнего обращения: 13.03.2024).

11. Python [Электронный ресурс]: режим доступа: <https://www.python.org/> (Дата последнего обращения: 13.03.2024).
12. Википедия Spring Framework [Электронный ресурс]: режим доступа: https://ru.wikipedia.org/wiki/Spring_Framework, свободный (Дата последнего обращения: 13.03.2024).
13. Habr Micronaut [Электронный ресурс]: режим доступа: <https://habr.com/ru/articles/418117/>, свободный (Дата последнего обращения: 13.03.2024).
14. Atlassian Scrum [Электронный ресурс]: режим доступа: <https://www.atlassian.com/ru/agile/scrum>, свободный (Дата последнего обращения: 13.03.2024).
15. DevOps [Электронный ресурс]: режим доступа: <https://efsol.ru/promo/scrum-devops/>, свободный (Дата последнего обращения: 13.03.2024).
16. Habr CI/CD [Электронный ресурс]: режим доступа: <https://habr.com/ru/companies/otus/articles/515078/>, свободный (Дата последнего обращения: 07.04.2024).
17. Habr VisualParadigm [Электронный ресурс]: режим доступа: <https://habr.com/ru/articles/685046/>, свободный (Дата последнего обращения: 13.03.2024).
18. Habr MVP [Электронный ресурс]: режим доступа: <https://habr.com/ru/companies/productstar/articles/508892/>, свободный (Дата последнего обращения: 07.04.2024).
19. Микросервисная архитектура [Электронный ресурс]: режим доступа: <https://www.atlassian.com/ru/microservices/microservices-architecture>, свободный (Дата обращения: 07.04.2024).
20. MVC паттерн статья Habr [Электронный ресурс]: режим доступа: <https://habr.com/ru/post/181772/>, свободный (дата последнего обращения: 13.03.2024). – Текст: электронный.

21. GitLab [Электронный ресурс]: режим доступа: <https://about.gitlab.com/>, свободный (Дата последнего обращения: 07.04.2024).
22. PostgreSQL – объектно-реляционная система управления базами данных [Электронный ресурс]: режим доступа: <https://web-creator.ru/articles/postgresql/>, свободный (Дата обращения: 07.04.2024).
23. Apache Kafka [Электронный ресурс]: режим доступа: <https://kafka.apache.org>, свободный (Дата обращения: 07.04.2024).
24. ZooKeeper [Электронный ресурс]: режим доступа: <https://zookeeper.apache.org/>, свободный (Дата последнего обращения: 07.04.2024).
25. JWT authentication: Best practices and when to use it. [Электронный ресурс]: режим доступа: <https://blog.logrocket.com/jwt-authentication-best-practices/>, свободный (Дата обращения: 07.04.2024).
26. Википедия DTO [Электронный ресурс]: режим доступа: <https://ru.wikipedia.org/wiki/DTO>, свободный (Дата обращения: 01.05.2024);
27. MapStruct [Электронный ресурс]: режим доступа: <https://mapstruct.org/>, свободный (Дата обращения: 01.05.2024).
28. Юнит-тесты [Электронный ресурс]: режим доступа: <https://habr.com/ru/articles/169381/>, свободный (Дата обращения: 01.05.2024);
29. Интеграционные тесты [Электронный ресурс]: режим доступа: https://logrocon.ru/news/intgration_testing, свободный (Дата обращения: 01.05.2024).
30. Диаграмма Ганта [Электронный ресурс]: режим доступа: <https://v8.1c.ru/platforma/diagramma-ganta/>, свободный (Дата обращения: 05.05.2024).
31. GanttPRO: Диаграмма Ганта для управления [Электронный ресурс]: режим доступа: <https://ganttpro.com/ru/>, свободный (Дата обращения: 05.05.2024).

32. Диграмма Ганта [Электронный ресурс]: режим доступа: <https://www.unisender.com/ru/blog/idei/chto-takoe-diagramma-ganta-i-kak-ee-sdelat/>, свободный (Дата обращения: 05.05.2024).

33. Гарантированные выплаты ППС [Электронный ресурс]: режим доступа: <https://www.mirea.ru/for-workers/personnel-work/garantirovannye-vyplaty-pps/>, свободный (Дата обращения: 05.05.2024).

34. Производственный календарь на 2024 г. [Электронный ресурс]: режим доступа: <https://www.consultant.ru/law/ref/calendar/proizvodstvennye/2024/>, свободный (Дата обращения: 05.05.2024);

35. Единый тариф страховых взносов на ОПС, ОСС, ОМС [Электронный ресурс]: режим доступа: https://www.consultant.ru/document/cons_doc_LAW_93256/977315fdb3b184feec5e6517d4866e602980ba76/, свободный (Дата обращения: 05.05.2024).

36. Федеральный закон от 24.07.1998 N 125-ФЗ [Электронный ресурс]: режим доступа: https://www.consultant.ru/document/cons_doc_LAW_19559/, свободный (Дата обращения: 05.05.2024).

37. Федеральный закон от 27.11.2023 N 549 [Электронный ресурс]: режим доступа: https://www.consultant.ru/document/cons_doc_LAW_462887/, свободный (Дата обращения: 05.05.2024).

38. Приказ Минтруда России от 30.12.2016 N 851н [Электронный ресурс]: режим доступа: https://www.consultant.ru/document/cons_doc_LAW_211247/, свободный (Дата обращения: 05.05.2024).

39. Тарифы на электроэнергию для населения и потребителей, приравненных к населению, на территории Москвы на 2024 год. Приказ ДЭПиР г. Москвы от 28.11.2023 № ДПР-ТР-188/23 [Электронный ресурс]: режим доступа: <https://www.mosenergosbyt.ru/individuals/>, свободный (Дата обращения: 05.05.2024).

ПРИЛОЖЕНИЕ А

Рисунки аналогов

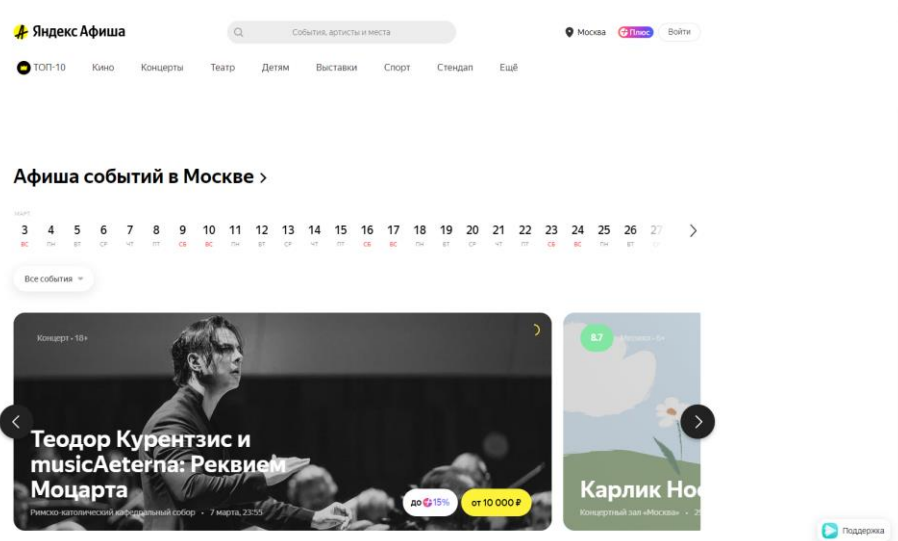


Рисунок А.1 – Яндекс афиша

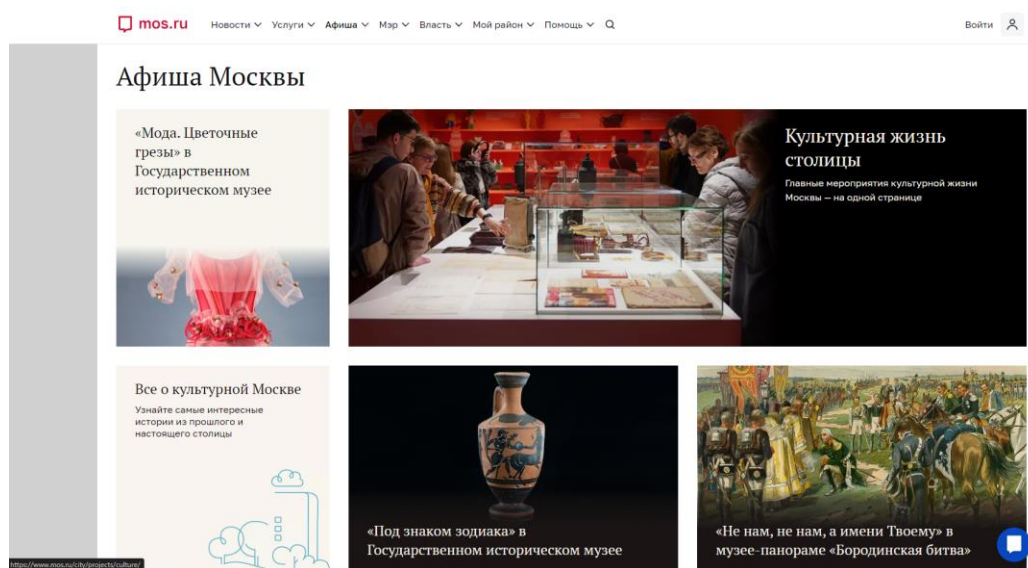


Рисунок А.2 – Мос.ру афиша

ПРИЛОЖЕНИЕ Б

Диаграммы вариантов использования

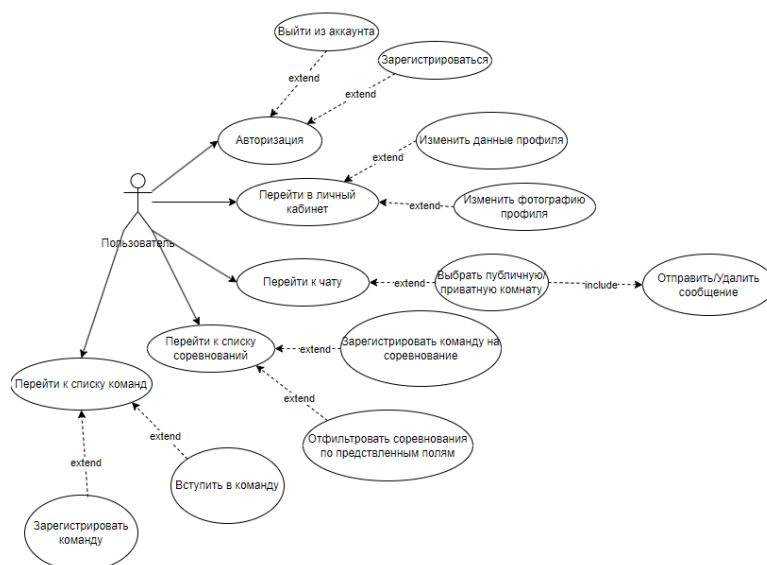


Рисунок Б.1 - Диаграмма вариантов использования пользователя
[разработано автором]

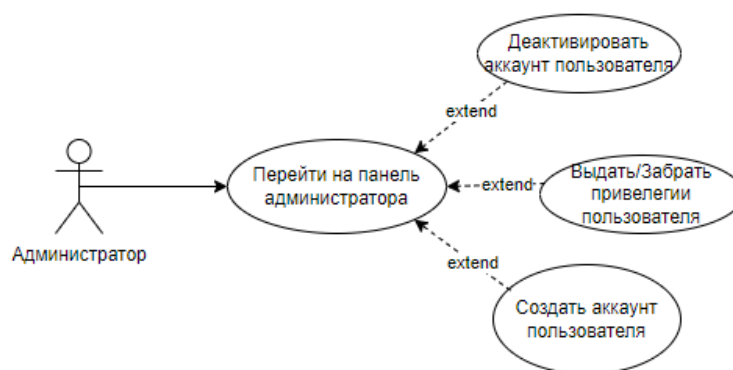


Рисунок Б.2 – Диаграмма вариантов использования администратора
[разработано автором]

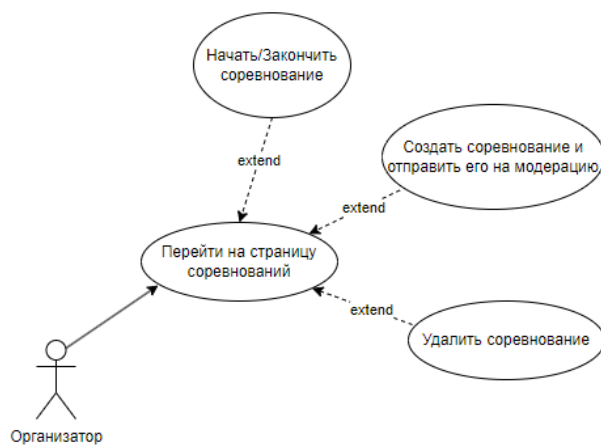


Рисунок Б.3 – Диаграмма вариантов использования организатора
[разработано автором]

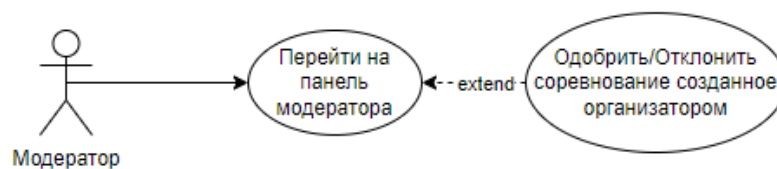


Рисунок Б.4 – Диаграмма вариантов использования модератора [разработано автором]

ПРИЛОЖЕНИЕ В

Макеты страниц

ИИспорт Авторизация

Изображение

Никнейм

Почта

Пароль

Регистрация

Логин

This wireframe shows a login page layout. At the top left is the text 'ИИспорт Авторизация'. On the left side is a large light blue rounded rectangle labeled 'Изображение'. On the right side is a white rounded rectangle containing three dark grey input fields labeled 'Никнейм', 'Почта', and 'Пароль' stacked vertically. Below these fields are two light blue buttons labeled 'Регистрация' and 'Логин'.

Рисунок В.1 – Макет страницы «Логин» [разработано автором]

ИИспорт Авторизация

Никнейм

Почта

Имя

Фамилия

Отчество

Телефон

Пароль

Регистрация

Изображение

This wireframe shows a registration page layout. At the top left is the text 'ИИспорт Авторизация'. On the left side is a white rounded rectangle containing seven dark grey input fields labeled 'Никнейм', 'Почта', 'Имя', 'Фамилия', 'Отчество', 'Телефон', and 'Пароль' stacked vertically. Below these fields is a light blue button labeled 'Регистрация'. On the right side is a large light blue rounded rectangle labeled 'Изображение'.

Рисунок В.2 – Макет страницы «Регистрация» [разработано автором]

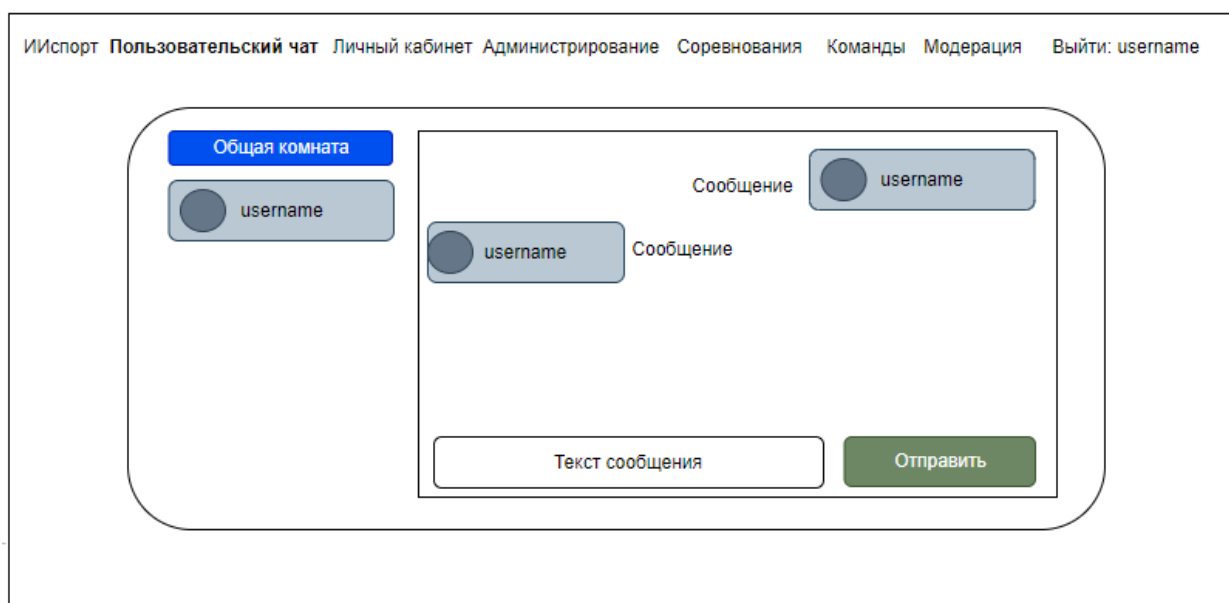


Рисунок В.3 – Макет страницы «Пользовательский чат» [разработано автором]

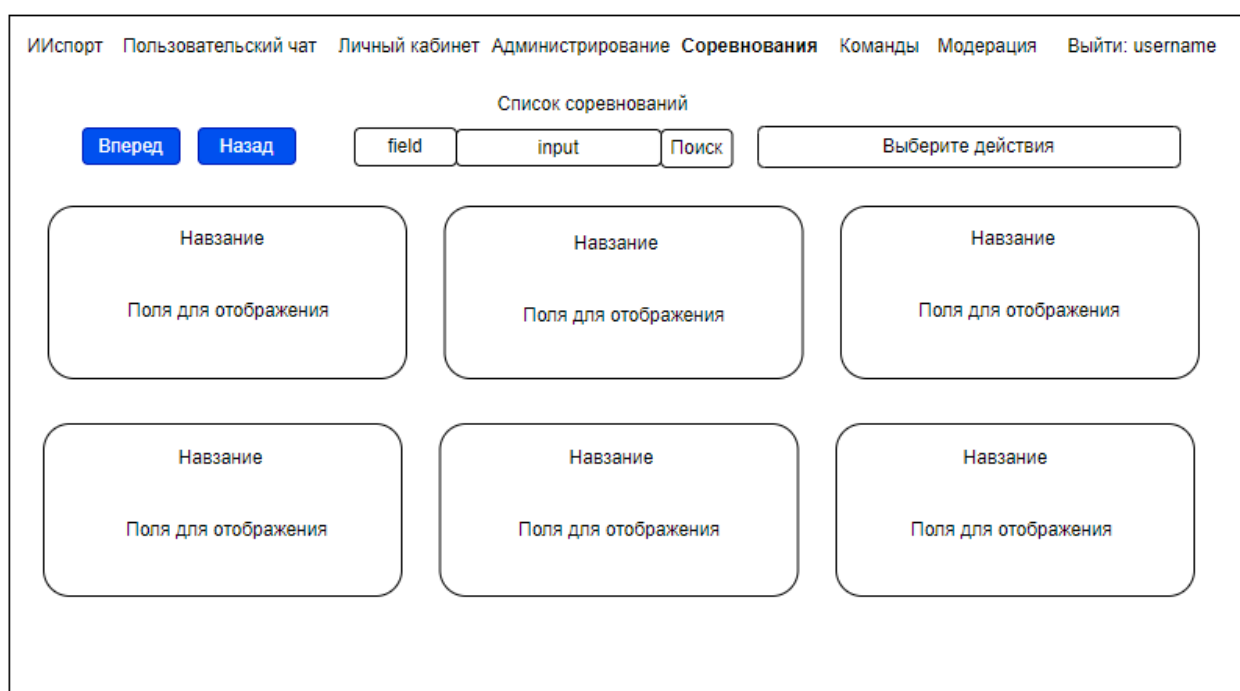


Рисунок В.4 – Макет страницы «Соревнования» [разработано автором]

ПРИЛОЖЕНИЕ Г

Листинги кода серверной части

Листинг Г.1 – Код модели User

```
@Getter
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Accessors(chain = true)
@EqualsAndHashCode
@Entity
@Table(name = "app_user")
@EntityListeners(AuditingEntityListener.class)
public class User implements UserDetails, Serializable {
    @Id
    @SequenceGenerator(name = "userIdSeq", sequenceName =
"user_id_seq", allocationSize = 1)
    @GeneratedValue(strategy =
 GenerationType.SEQUENCE, generator = "userIdSeq")
    private Long id;
    @Column(length = 64, unique = true)
    private String username;
    @Column(length = 64)
    private String email;
    @Column(length = 64)
    private String firstName;
    @Column(length = 64)
    private String lastName;
    @Column(length = 64)
    private String middleName;
    @Column(length = 128)
    private String fullName;
    @CreatedDate
    private LocalDateTime registrationDate;
    @LastModifiedDate
    private LocalDateTime updatedDate;
    @Column(length = 64)
    private String mobilePhoneNumber;
    @JsonIgnore
    @Column(length = 64)
    private String password;
    private boolean active;
    @JsonIgnore
    @ElementCollection(targetClass = Role.class, fetch =
 FetchType.EAGER)
    @CollectionTable(name = "user_role", joinColumns =
@JoinColumn(name = "user_id"))
    @Enumerated(EnumType.STRING)
```

Продолжение листинга Г.1

```
private Set<Role> authorities = new HashSet<>();

    @JsonBackReference
    @ManyToOne
    @JoinColumn(name = "team_id")
    private Team team;

    @OneToOne(targetEntity = Media.class, fetch =
FetchType.EAGER, cascade = CascadeType.ALL)
    @JoinColumn(name = "avatar_id", referencedColumnName =
"id")
    @JsonIgnoreProperties(value = "bytes", allowSetters =
true)
    private Media avatar;

    public User(String email, String username, String
password, Set<Role> authorities) {
        this.username = username;
        this.password = password;
        this.email = email;
        this.authorities = authorities;
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}
```

Листинг Г.2 – Код класса CrudServiceImpl

```
public abstract class CrudServiceImpl<E, CreateDto, PatchDto,
Dto, Id, R extends EntityRepository<E, Id>> extends
GetServiceImpl<E, Dto, Id, R> implements
CrudService<CreateDto, PatchDto, Dto, Id> {

    protected final EntityMapper<E, Dto, CreateDto, PatchDto>
entityMapper;

    protected CrudServiceImpl(R repository, EntityMapper<E,
Dto, CreateDto, PatchDto> mapper) {
        super(repository, mapper);
        this.entityMapper = mapper;
    }

    @Transactional
    @Override
    public Dto create(CreateDto createDto) {
        E entity =
entityMapper.toEntityFromCreateDto(createDto);
        entity = repository.save(entity);
        return mapper.domainToDto(entity);
    }

    @Override
    public Dto update(PatchDto patchDto, Id id) {
        E currentEntity = getEntityById(id);
        E entity =
entityMapper.toEntityFromCreateDtoPatch(currentEntity,
patchDto);
        entity = repository.save(entity);
        return mapper.domainToDto(entity);
    }

    @Override
    public void delete(Id id) {
        repository.deleteById(id);
    }
}
```

Листинг Г.3 – Код класса GetServiceImpl

```
public abstract class GetServiceImpl<E, DTO, ID, R extends
EntityRepository<E, ID>> implements GetService<DTO, ID> {

    protected final R repository;
    protected final BaseMapper<E, DTO> mapper;

    protected GetServiceImpl(R repository, BaseMapper<E, DTO>
mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    @Transactional(readOnly = true)
    @Override
    public Page<DTO> get(QuerySearchParams params) {
        Page<E> page = getEntities(params);
        return new PageImpl<>(
            mapper.domainsToDtos(page.getContent()),
            page.getPageable(),
            page.getTotalElements()
        );
    }

    protected Page<E> getEntities(QuerySearchParams params) {
        return
repository.findAll(RSQLJPASupport.rsq1(params.getQuery()), para
ms.getPageRequest());
    }

    @Transactional(readOnly = true)
    @Override
    public DTO getById(ID id) {
        E entity = getEntityById(id);
        return mapper.domainToDto(entity);
    }
    protected E getEntityById(ID id) {
        return repository.findById(id).orElseThrow(() -> new
EntityNotFoundException("EntityNotFound"));
    }
}
```

Листинг Г.4 – Код класса UserMapper

```
@Mapper(config = EntityMapperConfig.class)
public interface UserMapper extends EntityMapper<User,
UserDto, UserCreateDto, UserPatchDto> {
    @Override
    @Mappings({
        @Mapping(source = "teamId",target = "team.id")
    })
    User dtoToDomain(UserDto userDto);
    @Override
    @Mappings({
        @Mapping(source = "team.id",target = "teamId")
    })
    UserDto domainToDto(User domain);
    @Override
    @Mappings({
        @Mapping(target = "username", source =
"userPatchDto.username", defaultExpression =
"java(getDefaultValue(entity.getUsername(),
userPatchDto.getUsername()))"),
        @Mapping(target = "email", source =
"userPatchDto.email", defaultExpression =
"java(getDefaultValue(entity.getEmail(),
userPatchDto.getEmail()))"),
        @Mapping(target = "firstName", source =
"userPatchDto.firstName", defaultExpression =
"java(getDefaultValue(entity.getFirstName(),
userPatchDto.getFirstName()))"),
        @Mapping(target = "lastName", source =
"userPatchDto.lastName", defaultExpression =
"java(getDefaultValue(entity.getLastName(),
userPatchDto.getLastName()))"),
        @Mapping(target = "middleName", source =
"userPatchDto.middleName", defaultExpression =
"java(getDefaultValue(entity.getMiddleName(),
userPatchDto.getMiddleName()))"),
        @Mapping(target = "password", source =
"userPatchDto.password", defaultExpression =
"java(getDefaultValue(entity.getPassword(),
userPatchDto.getPassword()))"),
        User toEntityFromCreateDtoPatch(User entity, UserPatchDto
userPatchDto);
        default String getDefaultValue(String currentValue, String
newValue) {
            return newValue != null ? newValue : currentValue;
        }
    })
}
```

Листинг Г.5 – Код класса UserController

```
@RequestMapping(ApiPath.USER_API)
@CrossOrigin(origins = "http://localhost:3000")
public interface UserApi {

    @GetMapping
    @Operation(description = "Получение списка пользователей с
    фильтрацией")
    Result<List<UserDto>> getUser(
        @Parameter(description = "query - содержит условия
    для фильтрации")
        QuerySearchParams params
    );

    @GetMapping("/{id}")
    @Operation(description = "Получение пользователя по
    идентификатору")
    ResponseEntity<UserDto> getUserById(
        @Parameter(description = "Идентификатор
    пользователя")
        @PathVariable Long id
    );

    @PostMapping
    @Operation(description = "Создание пользователя")
    UserDto createUser(
        @Parameter(description = "Данные пользователя")
        @RequestBody UserCreatedDto userDto
    );

    @PatchMapping("/{id}")
    @Operation(description = "Обновление пользователя")
    ResponseEntity<?> updateUser(
        @Parameter(description = "Данные пользователя")
        @RequestParam(value = "user", required = false)
        String jsonUser,
        @PathVariable Long id,
        @RequestParam(value = "file", required = false)
        MultipartFile multipartFile
    ) throws IOException;

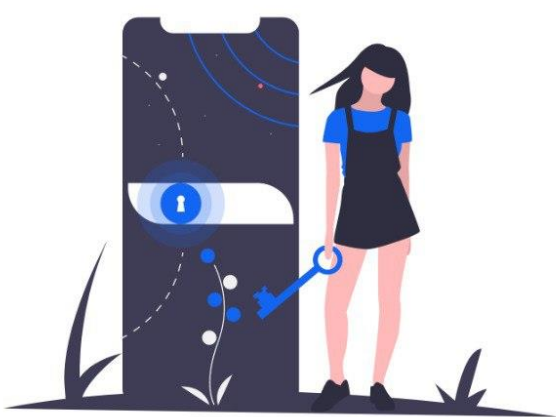
    @GetMapping("/{id}/roles")
    @Operation(description = "Получение ролей пользователя")
    ResponseEntity<List<Role>> getRoles(
        @Parameter(description = "Id пользователя")
        @PathVariable Long id
    );

    @PostMapping("/role/{id}")
    @Operation(description = "Выдать роль пользователю")
    Set<Role> setRole(
        @Parameter(description = "идентификатор
    пользователя") @PathVariable Long id,
```

ПРИЛОЖЕНИЕ Д

Итоговый вид страниц приложения

ИИспорт Авторизация



Никнейм

Почта

Пароль

[Регистрация](#)
[Логин](#)

Рисунок Д.1 – Страница «Логин» [разработано автором]

localhost:3000/chat

ИИспорт Пользовательский чат Личный кабинет Администрирование Соревнования Модерация Команды Выйти: Nikita

Общая комната

Nikita

Привет! Nikita

enter the message

send

Рисунок Д.2 – Страница «Пользовательский чат» [разработано автором]

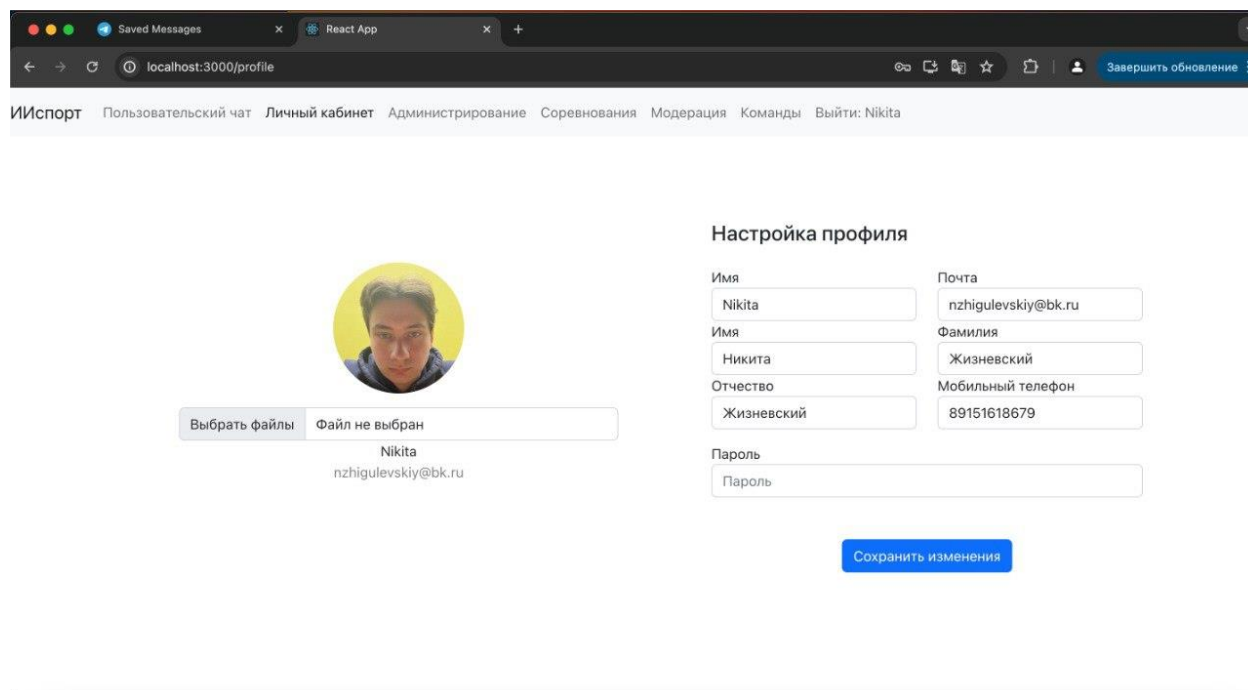


Рисунок Д.3 – Страница «Личный кабинет» [разработано автором]

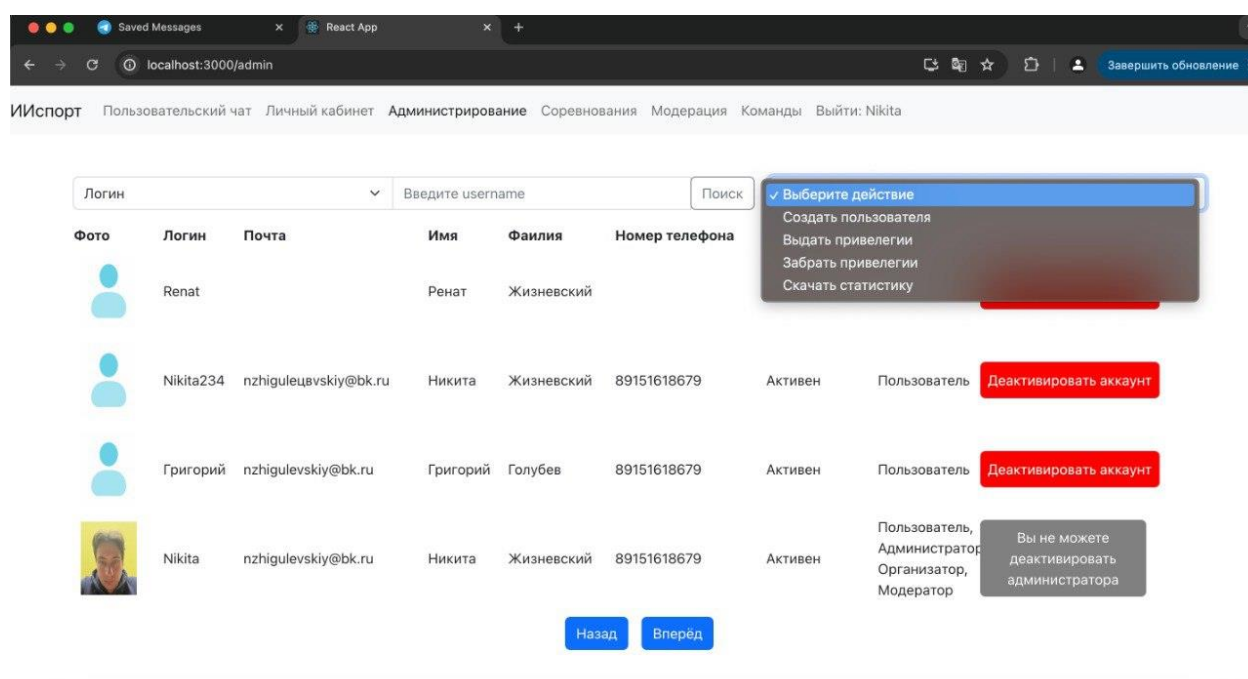


Рисунок Д.4 – Страница «Администрирование» [разработано автором]

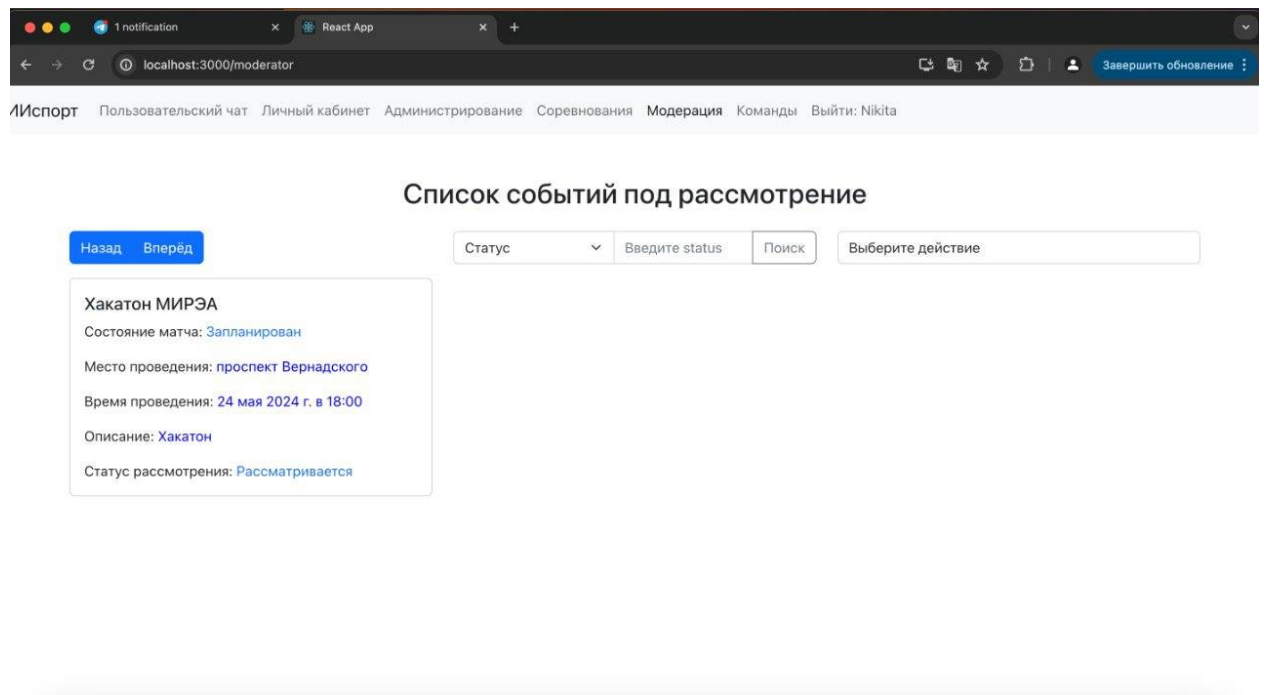


Рисунок Д.5 – Страница «Модерация» [разработано автором]

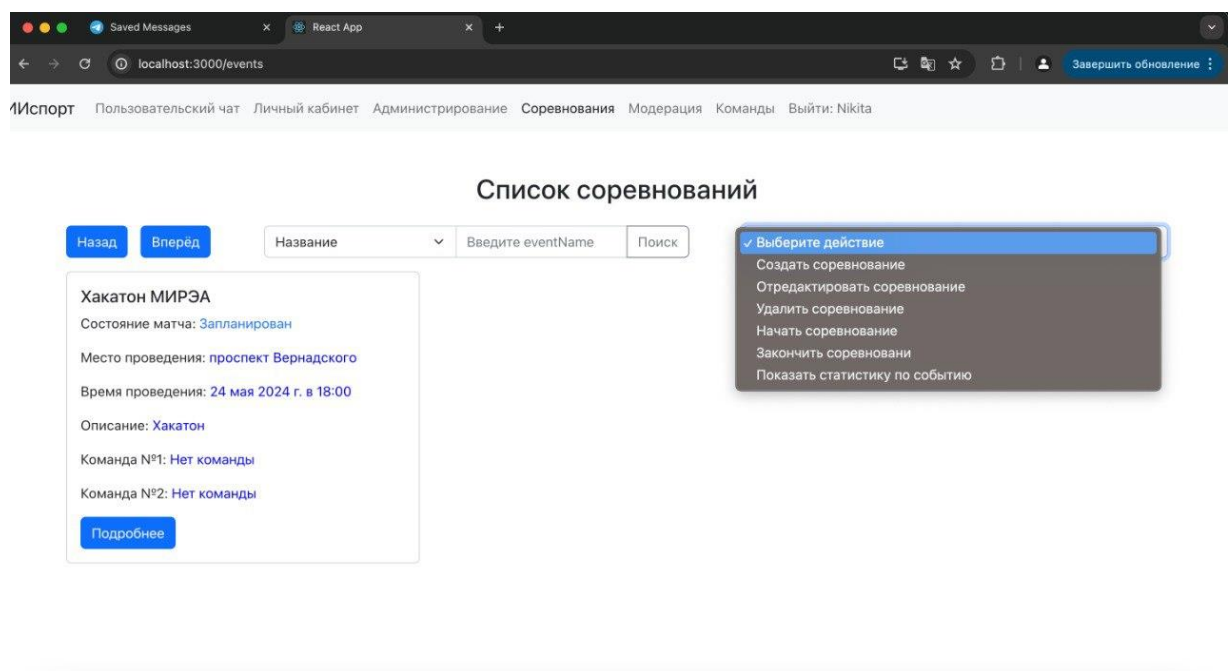


Рисунок Д.6 – Страница «Соревнования» [разработано автором]

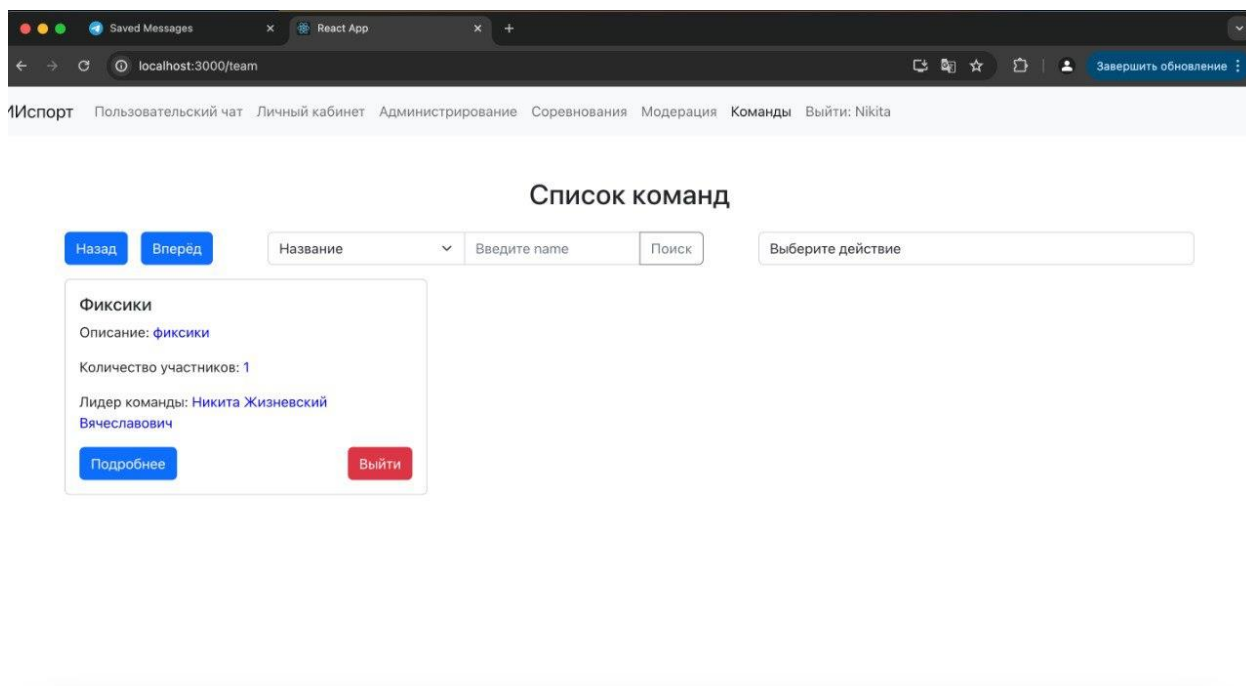


Рисунок Д.7 – Страница «Команды» [разработано автором]

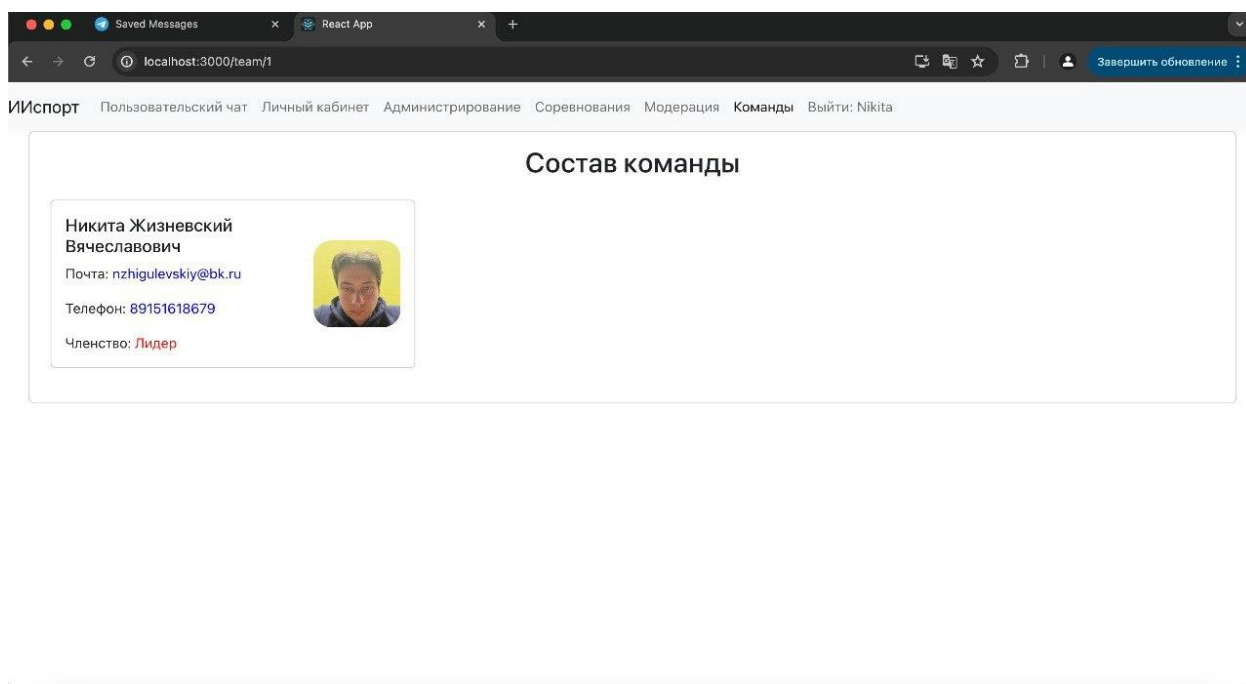


Рисунок Д.8 – Страница «Состав команды» [разработано автором]

ПРИЛОЖЕНИЕ Е

Диаграмма Ганта

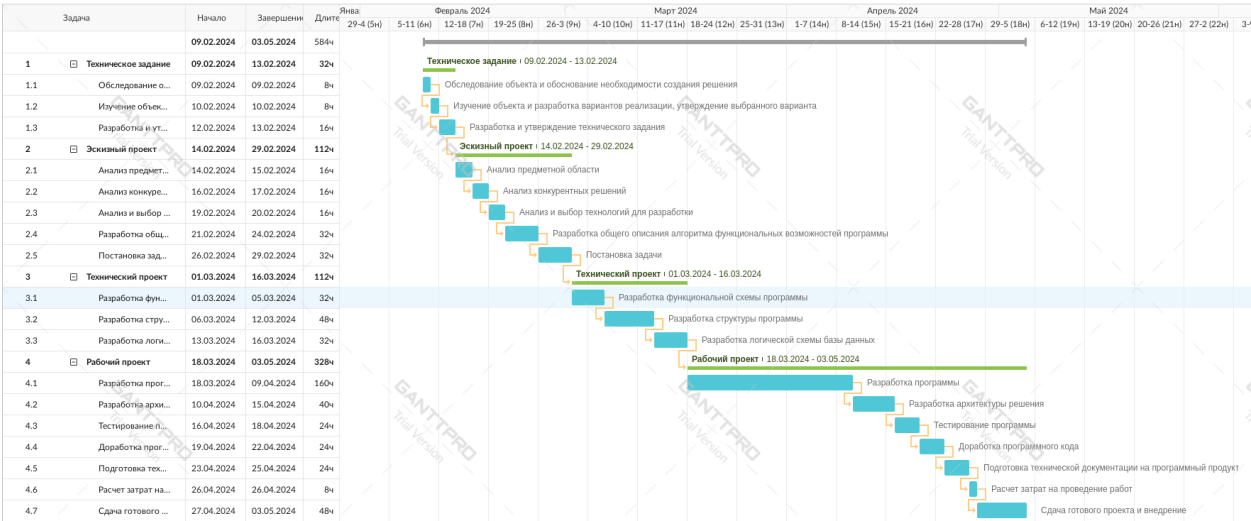


Рисунок Е.1 – Диаграмма Ганта [разработано автором]